A PEDAGOGICAL PAPER OF THE SERIES ON AUTONOMOUS SYSTEMS IN FINANCE

FROM PREDICTION TO REASONING TO AGENCY

How Generative AI Evolved from Next-Token Continuation to Deliberative Models and Governed Autonomous Systems

PREDICT • ADAPT • DELIBERATE • USE TOOLS • ACT

"Prediction supplies possibility. Reasoning disciplines possibility. Agency converts selected possibilities into action—and governance defines what action is legitimate."

ALEJANDRO REYNOSO

JUDGE BUSINESS SCHOOL. UNIVERSITY OF CAMBRIDGE.JULY 2026

From Prediction to Reasoning to Agency

This monograph is an educational and technical synthesis built around three transparent, Colab-scale experiments: *Sherlock Holmes Next-Sentence Transformer*, *Training a Holmesian Reasoning Model with QLoRA*, and a simplified *Autonomous Mystery Investigator*. The third experiment was executed against the fictional case using bounded, simulated tool interfaces; the larger architecture in Chapter 7 is the reference design toward which that notebook implementation points.

The mystery used in the case study is fictional. It is designed to illustrate system architecture, evidence discipline, and governance. It is not an allegation about any real person or institution.

First edition, July 2026.

Abstract

Generative artificial intelligence is often narrated as a succession of revolutions: first language models predicted text, then reasoning models solved problems, and finally agents began to act. That account captures the change in user experience but obscures the engineering continuity underneath it. This monograph develops a cumulative progression from probabilistic continuation to task adaptation, structured deliberation, tool use, and governed agency.

The argument is developed through three linked experiments. A small causal Transformer is first trained from scratch on public-domain Sherlock Holmes stories, making next-token learning and sampling directly observable. Its continuations can be stylistically convincing, but fluency alone does not establish deduction. A pretrained instruction model is then adapted with QLoRA to follow an explicit reasoning contract: identify observations, separate fact from inference, preserve competing hypotheses, expose contradictions, recommend discriminating tests, and calibrate a conclusion. Finally, a simplified multi-agent loop is implemented and run against a fictional mystery. The execution produces a stateful trajectory, gate decisions, ledger updates, and a provisional report; Chapter 7 presents a condensed trace and distinguishes the executed notebook from the fuller reference architecture.

The experiments support a precise conceptual claim. Reasoning does not replace generation; it organizes generation and computation around intermediate states and independent criteria. Agency does not reside in model weights alone; it is constructed by coupling models to goals, tools, state, feedback, permissions, and stopping rules. Each addition enlarges both capability and consequence.

The resulting contribution is pedagogical and architectural. It explains not only how modern systems became more capable, but also why their evaluation must widen from output quality to reasoning validity, action trajectories, authorization, recoverability, and institutional legitimacy. The architecture of intelligence must therefore be accompanied by an architecture of authority.

The thesis in one sentence

Prediction supplies linguistic possibility; reasoning organizes possibilities under constraints; agency converts selected possibilities into state-changing action. Governance must expand at every step because the radius of consequence expands with capability.

Preface

This book began with a simple pedagogical ambition: make a Transformer learn the language of Sherlock Holmes and observe what it predicts next. That experiment is valuable precisely because it is modest. The model does not receive a detective's notebook, a symbolic logic engine, or a theory of evidence. It receives text, a causal mask, and a loss function. Yet it learns regularities strong enough to produce plausible continuations. The result provides a window into the generative substrate of modern language models.

The next question is unavoidable. If the model can sound like Holmes, can it reason like Holmes? The answer requires a change in the training object. Narrative prose contains demonstrations of conduct, dialogue, suspense, and explanation, but it does not reliably expose a complete, machine-readable sequence of observation, abduction, testing, revision, and conclusion. To teach such a method, one must construct explicit reasoning episodes, prevent solution leakage, create counterexamples, supervise intermediate behavior, and test generalization beyond the original stories.

The third question reaches beyond model training. Suppose both artifacts exist: a narrative generator and a structured reasoner. Can they be organized as agents to solve a mystery not present in either training corpus? The third notebook tests that question in a bounded Colab execution using a fictional case and simulated tool returns. It records the resulting trajectory and provisional report while also exposing what a notebook-scale run cannot establish about production security, live credentials, legal approval, or institutional deployment.

Sherlock Holmes is therefore not ornamental. The detective story gives us a controlled theater in which three different capabilities can be separated:

1. producing a plausible continuation;
2. deriving a warranted conclusion from evidence; and
3. conducting a bounded investigation through action.

The monograph is intended for advanced students, practitioners, executives, and researchers who want a rigorous explanation without losing the intuitive thread. Equations are included where they clarify the mechanism; diagrams and tables carry the architecture; and each major concept is tied back to the three experiments.

The aim is not to anthropomorphize software. It is to understand exactly which computational and institutional ingredients create the appearance—and sometimes the useful reality—of increasingly autonomous intelligence.

The book follows the ladder in order. Chapters 1–3 establish the predictive substrate and the post-training bridge to task performance. Chapters 4 and 5 examine deliberation and the Holmesian reasoning experiment. Chapters 6 and 7 move from tool use to governed agency. Chapters 8 and 9 widen the analysis to evaluation and governance. Chapter 10 draws the progression together and identifies the limits of the demonstration.

Two editorial conventions make the evidentiary boundary explicit. Each experiment begins with a *Demonstrated vs. specified* flag. Here, *demonstrated* means implemented and executed at notebook scale; it does not by itself imply statistical generalization, production validity, or independent replication. Equations that express an ideal rather than a directly computed notebook mechanism carry an *Implementation status* note identifying them as implemented, schematic, qualitatively approximated, or normative. Worked calculations in Chapters 4 and 8, with executable code in Appendix A, allow readers to reproduce and challenge the principal quantitative ideas even where the notebooks use qualitative controls.

Reader's Map

Table 1: How to use the monograph

Reader	Recommended route	Central question
Executive	Chapters 1, 6, 9, and 10	What changes when a model becomes an operational agent?
Student	Chapters 1–8 in sequence	Which technical additions create each rung of the ladder?
Model builder	Chapters 2–5 and Appendix A	How do training objectives, adapters, verifiers, and evaluations differ?
Governance lead	Chapters 6–10	Where must permissions, evidence, audit, and human review enter?
Researcher	Chapters 4, 5, 8, and References	What is demonstrated, and what remains scientifically unproven?

A note on terminology

This monograph uses *reasoning model* operationally: a language model trained or prompted to allocate additional computation to multi-step problem solving, hypothesis testing, verification, or search. It does not claim that the model reasons in the same phenomenological sense as a human being. Likewise, *agent* denotes a goal-directed software system with a model inside an action loop; it does not imply consciousness, moral personhood, or unrestricted autonomy.

Contents

List of Figures

List of Tables

1

The Progression Ladder

Chapter compass

Argument. The apparent shifts from prediction to reasoning to agency are cumulative expansions around a persistent generative core.
Reader outcome. Distinguish models, deliberative systems, tool-using agents, and governing institutions—and see why the object of evaluation widens at every stage.

1.1 Three revolutions, one continuous substrate

The popular history of generative AI is often told as a succession of replacements. Models first predicted text, then learned to reason, and finally became agents that could act. The sequence is memorable because the user experience did change dramatically. It is technically misleading, however, if it suggests that each stage discarded the machinery of the one before it.

A modern reasoning model still generates tokens. An agent still relies on model outputs to represent plans, select tools, interpret observations, and decide whether work is complete. What changes is the *computational and institutional context* in which generation occurs. Instructions reshape the task; post-training reshapes preferred behavior; search and verification reshape inference; tools connect the model to external systems; state gives decisions continuity; and permissions determine which proposed actions may become real.

Figure 1.1 therefore presents a ladder of cumulative construction. Each rung retains the capacities below it while adding new mechanisms, new failure modes, and a wider radius of consequence.

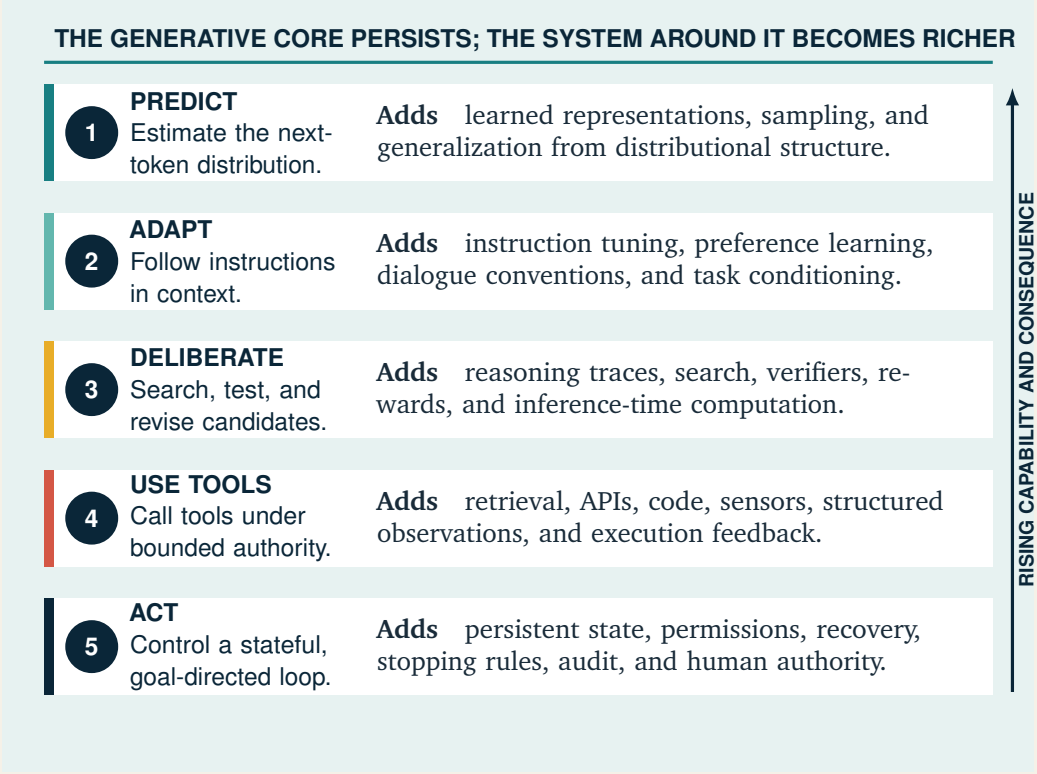

Figure 1.1: The generative AI progression ladder. The rungs are cumulative.

1.2 The five rungs

The rungs identify where additional structure enters the system. They are analytical distinctions rather than claims that products evolve through one universal implementation path.

Rung 1: Predict. A causal language model estimates a conditional probability distribution over the next token. When repeatedly sampled, the model produces paragraphs, code, dialogue, or stories. The training objective does not explicitly contain grammar, characters, or deduction, yet the model must learn useful representations of them to reduce prediction error.

Rung 2: Adapt. Pretraining creates a general conditional generator; instruction tuning and preference optimization shape it into an assistant. The user can now specify a task in natural language. Few-shot examples and long contexts can further adapt behavior without changing the model's weights. This rung transformed language models from text completion engines into general-purpose interfaces.

Rung 3: Deliberate. The system allocates computation to intermediate structure: decompositions, candidate hypotheses, calculations, searches, critiques, or verification. Deliberation can be elicited through prompting, learned through supervised reasoning demonstrations, encouraged through reinforcement learning, or expanded through inference-time search. The central advance is not verbosity. It is the disciplined use of intermediate state to improve a final answer.

Rung 4: Use tools. The model can request external operations. A calculator supplies arithmetic reliability; retrieval supplies current or proprietary knowledge; a code interpreter executes formal procedures; an API exposes an application; a sensor observes an environment. Tool results return as observations that can alter the next decision. The system is no longer closed within the model's parameters and prompt.

Rung 5: Act. An agent is given a goal and allowed to choose a sequence of steps under constraints. It maintains task state, selects tools, observes results, handles exceptions, and recognizes completion or escalation conditions. Multiple agents may specialize and collaborate under an orchestrator. Because actions can change records, send messages, spend money, or affect people, permissions and audit become first-class design elements.

1.3 A capability is not an architecture

Three distinctions prevent conceptual confusion.

1.3.1 A model is not an agent

A model maps an input context to an output distribution. An agent is a software system that uses a model to control part of a workflow. The agent includes at least an objective, an execution loop, state, and some action interface. In practice it also needs guardrails, permissions, observability, error handling, and stopping conditions.

OpenAI's practical definition similarly excludes applications in which the language model does not control workflow execution (OpenAI, 2025).

1.3.2 A chain of thought is not proof of reasoning

A fluent intermediate explanation can be correct, mistaken, irrelevant, or reverse-engineered after the answer. Reasoning quality requires external criteria: mathematical validity, causal consistency, evidence attribution, counterfactual sensitivity, successful execution, or another verifier. Process supervision can help, but the text of a rationale should not automatically be treated as a faithful transcript of internal computation (Lightman et al., 2023).

1.3.3 Autonomy is not binary

Autonomy varies along dimensions:

- **scope**: one subtask versus an end-to-end objective;
- **duration**: one inference versus a long-running workflow;
- **authority**: read-only retrieval versus irreversible action;
- **adaptation**: fixed steps versus replanning after observations;
- **initiative**: user-invoked versus event-triggered behavior;
- **oversight**: approval at every step versus exception-only review.

It is therefore more precise to ask *which decisions are delegated, over which resources, for how long, under whose authority, and with what reversibility* than to ask whether a system "is autonomous."

1.4 The widening object of evaluation

As the ladder rises, what must be evaluated expands.

The final row is essential. A technically successful agent may still be institutionally unacceptable. A system might reach a correct internal conclusion while violating privacy, due process, labor rules, evidentiary standards, or legitimate authority. Evaluation must therefore ask not only whether the system worked, but whether it worked in a way that an institution may responsibly defend.

1.5 Sherlock Holmes as a controlled teaching device

The Holmes corpus is unusually useful for this ladder because it supports three tasks that appear similar on the surface but are computationally distinct.

1. **Continuation:** What sentence is plausible after this passage?
2. **Inference:** Which hypothesis is best supported by the available clues?
3. **Investigation:** What evidence should be sought next, which tool should obtain it, and when should the system stop?

Table 1.1: The object of evaluation changes at each stage

Stage	Primary object	Typical metrics	Characteristic failure
Generator	Output distribution	loss, perplexity, calibration, diversity, factuality	plausible but false continuation
Reasoner	Solution trajectory	answer accuracy, step validity, evidence grounding, robustness	convincing but invalid rationale
Agent	Action trajectory	task success, tool correctness, recovery, cost, safety, authorization	compounding or unauthorized action
Institution	Sociotechnical outcome	accountability, contestability, legal compliance, distributional impact	locally correct system causing unacceptable consequences

The first can be approached with causal language modeling. The second requires an explicit evidentiary protocol and evaluation on hidden or held-out cases. The third requires an architecture that can acquire observations and act outside the model while remaining bounded by policy. These distinctions form the conceptual spine of the book.

The principle of cumulative construction

Generation is a capability of a model. Reasoning is a disciplined way of allocating generation and computation. Agency is a control architecture that converts model outputs into observations and actions. The ladder rises by adding structure, not by escaping probability.

2

Rung One: The Predictive Machine

Chapter compass

Argument. Next-token prediction is a local objective with a demanding global consequence: accurate prediction across diverse text requires compressed representations of linguistic and worldly structure.
Reader outcome. Understand the causal objective, the Transformer mechanism, decoding choices, and the boundary between plausible continuation and warranted inference.

2.1 The deceptively simple objective

The first rung begins with an objective that is easy to state and easy to underestimate. The model is not directly instructed to learn grammar, facts, code, genres, or argument. It is asked to predict what comes next.

Let a tokenized sequence be $x_1, x_2, \ldots, x_T$. A causal language model factorizes the probability of the sequence as

$$p_\theta(x_{1:T}) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}), \tag{2.1}$$

where $x_{<t}$ denotes all tokens before position t, and θ denotes the model parameters. Training minimizes the negative log-likelihood, usually expressed as token-level cross-entropy:

$$\mathcal{L}(\theta) = -\frac{1}{T} \sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t}). \tag{2.2}$$

At every training position, the model faces a local question: given the preceding context, how much probability should be assigned to the token that actually occurred? The global sophistication of the resulting system arises from the difficulty of answering that question well across vast and heterogeneous data. Accurate

prediction rewards representations of spelling, syntax, semantics, discourse, genre, code, argument, entities, and recurring regularities in the world described by the data.

The phrase *next-sentence predictor* is therefore pedagogically useful but technically approximate. The objective predicts the next *token*. A sentence is produced through repeated token selection until punctuation or a special stopping token appears. This distinction matters: the model is not selecting a complete sentence from a catalog. It constructs one sequentially.

2.2 Why the Transformer mattered

Earlier sequence models used recurrent computation, processing tokens step by step through a hidden state. The Transformer replaced recurrence with attention-based interactions and made parallel training far more effective (Vaswani et al., 2017). For one attention head, the central computation is

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V, \tag{2.3}$$

where Q, K, and V are learned query, key, and value projections; d_k scales the dot products; and M is a causal mask that prevents a token from seeing the future.

The equation can be read intuitively:

1. each token asks a learned question through its query;
2. it compares that query with keys produced by other earlier tokens;
3. the comparison scores become attention weights;
4. the token gathers a weighted combination of value vectors;
5. multiple heads learn different relational patterns in parallel.

Stacked attention and feed-forward blocks repeatedly transform the representation of each position. Positional information tells the model where tokens occur. Residual connections and normalization make deep optimization tractable. The final representation is projected into vocabulary logits, which are converted into probabilities by softmax.

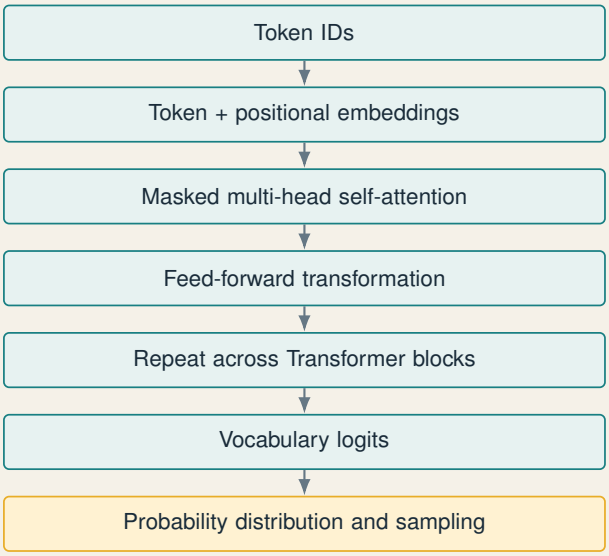

Figure 2.1: A decoder-only causal Transformer in conceptual form.

2.3 Learning as compression

A useful, though incomplete, interpretation of language modeling is probabilistic compression. If a model assigns high probability to the observed data, fewer bits are needed to encode those observations relative to its distribution. The model improves by discovering regularities that reduce surprise. Perplexity,

$$\mathrm{PPL} = \exp(\mathcal{L}), \tag{2.4}$$

can be interpreted loosely as the effective number of equally likely choices faced by the model at each position. Lower perplexity on held-out data usually indicates better predictive fit, but it is not a universal measure of usefulness, truthfulness, or safety.

Scaling work showed systematic relationships between model loss and model size, dataset size, and compute (Kaplan et al., 2020). Subsequent research refined the allocation of compute between parameters and training tokens (Hoffmann et al., 2022). Scaling increased not only fluency but also the ability to perform tasks from prompts and examples (Brown et al., 2020). Yet the objective remained next-token likelihood.

This continuity is one of the central facts of modern AI. Translation, summarization, question answering, and code generation can all be expressed through conditional sequence generation. That observation does not make the tasks trivial or the learned representations shallow. It explains the leverage of a broad training distribution combined with a flexible language interface.

2.4 Experiment I: the Holmes continuation model

Demonstrated vs. specified

Implemented and executed: corpus acquisition and cleaning, word-level tokenization, causal Transformer training, validation loss and perplexity, next-token inspection, and temperature/top-k/top-p sampling. **Specified but not claimed:** production-scale language competence, broad factual reliability, or deductive reasoning.

The first notebook makes this rung deliberately transparent. Rather than beginning with a large pretrained model, it trains a compact causal Transformer from scratch on five public-domain works by Arthur Conan Doyle obtained from Project Gutenberg:

- *The Adventures of Sherlock Holmes*;
- *The Return of Sherlock Holmes*;
- *The Memoirs of Sherlock Holmes*;
- *The Hound of the Baskervilles*;
- *A Study in Scarlet*.

The texts are cleaned conservatively, segmented, and tokenized with a transparent word-level tokenizer. Beginning- and end-of-sentence tokens expose boundaries. The corpus is divided within each book so that later material is held out for validation. Training windows preserve causal order. A small Transformer with token embeddings, positional embeddings, masked attention, feed-forward layers, normalization, and a vocabulary projection is trained from scratch with AdamW, gradient clipping, and a scheduled learning rate.

What the notebook demonstrates

- A Transformer can learn usable linguistic regularities without explicit grammar rules.
- Training loss and validation perplexity expose whether predictive fit is improving.
- Temperature, top-k, and top-p sampling change the diversity-risk tradeoff.
- Inspecting the immediate next-token distribution makes probabilistic generation visible.
- A reproducibility manifest and saved vocabulary turn the experiment into an auditable artifact.

2.5 Sampling is a policy over possibilities

Once the model produces logits z_i for vocabulary item i, temperature $\tau > 0$ modifies the distribution:

$$p_i(\tau) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}. \tag{2.5}$$

Low temperature sharpens the distribution, favoring common continuations; high temperature flattens it, increasing novelty and error. Top-k sampling retains only the k most probable tokens. Top-p, or nucleus sampling, retains the smallest set whose cumulative probability exceeds p. These methods do not improve the model's knowledge. They choose how to navigate the probability landscape it has learned.

For a mystery story, some randomness is desirable. A deterministic generator may repeat conventional phrases; an excessively random generator may break syntax, continuity, or facts. Generation quality is therefore partly a decoding problem.

2.6 What has and has not been learned

The model may produce a continuation such as:

> Holmes bent over the mark beside the window, then turned without a word toward the lamp.

This can be stylistically and narratively plausible. But the following claims would exceed the evidence:

- the model has acquired Holmes's method of deduction;
- it understands which details are legally or scientifically probative;
- it can distinguish a clue from a decorative narrative detail;
- it will revise beliefs coherently when contradictory evidence appears;
- its continuation reports what actually happened.

The model has learned a distribution over textual patterns. Those patterns may encode fragments of reasoning because human texts contain reasoning. Still, a continuation can imitate the surface form of an explanation without satisfying its inferential obligations.

2.7 The prediction paradox

The success of next-token training creates a productive paradox. The objective appears too local to yield broad competence, yet broad competence can emerge. That tension invites two symmetrical errors.

The first is **reductionist dismissal**: "It merely predicts the next word, so nothing interesting is happening." This ignores the representational burden of accurate prediction across complex data. The second is **anthropomorphic inflation**: "It

writes a deduction, so it possesses a detective's understanding." This confuses behavioral resemblance with a validated reasoning process.

A more scientific position holds both facts at once. Next-token prediction is a concrete optimization objective, and learning to perform it well can produce rich representations and broad behavioral competence. Neither the objective nor the observed behavior settles questions of understanding or reliability in advance. Those questions must be tested task by task. The next rung asks how post-training can turn this broad generator into a more dependable interface for user intent.

Lesson of Rung One

The predictive model learns what language and narratives make likely. It can generate a possible next sentence. It does not, by that fact alone, establish which hypothesis the evidence warrants.

3

Rung Two: From Completion to Task Performance

Chapter compass

Argument. Post-training turns a broad conditional generator into a usable assistant by reshaping which continuations are preferred, without replacing the predictive substrate.
Reader outcome. Separate weight-changing adaptation from prompting and retrieval, and understand why structured outputs create inspectable commitments rather than guaranteed truth.

3.1 Why pretraining is not enough

A model trained on heterogeneous text can acquire broad competence without acquiring a stable obligation to the person using it. When given a question, a raw pretrained model may answer, invent a second question, imitate a forum thread, continue a quoted passage, or reproduce another pattern licensed by its data. The objective itself does not specify that the model should be helpful, follow instructions, disclose uncertainty, or refuse unsafe requests.

This gap separates **capability acquisition** from **behavioral adaptation**. Pretraining builds a broad representational and generative substrate. Post-training turns that substrate into an interface with learned conventions about tasks, dialogue, and acceptable behavior.

3.2 Instruction tuning

In supervised instruction tuning, examples pair an instruction u with a desired response y. The model is optimized to produce the response conditional on the instruction and perhaps a system policy and dialogue history:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\sum_{t} \log p_\theta(y_t \mid u, y_{<t}). \tag{3.1}$$

The objective is still next-token likelihood, but the training distribution is different. It contains demonstrations of how a helpful assistant should transform user intent into an answer. The model learns conventions such as explaining, formatting, asking for clarification, and following constraints.

Instruction tuning is a bridge because it turns natural language into a practical medium for task specification. Users no longer need to train a separate model for every application. They can describe an objective, provide demonstrations, impose a schema, attach evidence, and state constraints in the same medium the model already processes.

3.3 Preference optimization and human intent

Several responses can be plausible under the language-model objective while differing greatly in usefulness. Preference data ranks candidates. In the InstructGPT pipeline, supervised fine-tuning was followed by a reward model trained on human rankings and reinforcement learning to optimize behavior under that reward (Ouyang et al., 2022). The study's central lesson was that scale alone does not guarantee instruction following or alignment with user intent.

This stage changes which continuations the system prefers; it does not change the fact that responses are generated token by token. Preference optimization adds a learned normative layer: among many linguistically plausible responses, favor those judged more useful, truthful, safe, or policy-compliant under the training procedure. Yet a reward model estimates patterns in labeler comparisons; it is not a direct measure of truth, welfare, or universal human intent. The quality of the layer depends on whose preferences are represented, how disagreements are resolved, which proxy behaviors are rewarded, and whether optimization generalizes beyond the comparison distribution.

3.4 In-context learning

Large pretrained models can also adapt at inference time. A prompt may contain a few demonstrations:

```
Observation: The window is broken from the inside.
Label: FACT

Observation: The intruder entered through the window.
Label: INFERENCE

Observation: The clock stopped at 9:10.
Label:
```

The model continues with a label by inferring the local task pattern. No gradient update occurs. This phenomenon, often called in-context learning, became especially visible at scale (Brown et al., 2020). It is powerful but fragile: behavior depends on example choice, ordering, wording, and the model's ability to use long context. Relevant information can be underused when buried in the middle of long prompts (Liu et al., 2024).

3.5 Retrieval is context, not learning

Another common confusion is to call every supplied document "training." Retrieval-augmented generation does not normally change model weights. A retrieval system finds passages and inserts them into the model's context; the model then conditions its response on those passages (Lewis et al., 2020).

Four diagnostic questions keep the mechanisms distinct: Did any parameter change? Where did the new information enter? How long does the change persist? Can the system only answer, or can it also alter external state? The first three questions separate training, prompting, and retrieval; the fourth anticipates the boundary between a model and an agent.

Table 3.1: Four distinct ways to change model behavior

Mechanism	Weights change?	What it supplies	Typical persistence
Pretraining	Yes	broad language and world patterns	base model lifetime
Fine-tuning	Yes, fully or through adapters	domain behavior or task protocol	adapted model lifetime
Prompting	No	instructions and examples	current inference or conversation
Retrieval	No	external evidence or current data	current inference unless stored

This distinction will matter in the Holmesian reasoner. QLoRA changes a small set of adapter parameters. The evidence ledger for a new case is supplied at inference time. A tool result is an observation, not a permanent update to the model.

3.6 Structured outputs as epistemic scaffolding

Natural-language answers are flexible but difficult to validate. A stable schema can require explicit fields:

```
{
  "observations": [{"evidence_id": "E1", "claim": "..."}],
  "hypotheses": [
    {
```

```
    "claim": "...",
    "support": ["E1", "E3"],
    "contradictions": ["E5"],
    "prediction": "..."
   }
 ],
 "best_next_test": "...",
 "conclusion": "...",
 "confidence": 0.63
}
```

The schema does not guarantee sound reasoning. It does, however, create inspectable commitments. A verifier can detect a nonexistent evidence identifier, missing alternatives, out-of-range confidence, or an unparsable response. Structure converts part of the problem from subjective prose evaluation into deterministic checking. It does not make every field equally meaningful: syntactic validity is directly testable, evidence references can be checked against a ledger, but causal adequacy and conclusion quality still require substantive evaluation.

3.7 The assistant is still not a reasoner or an agent

An instruction-following model can execute multi-step tasks in one response and may display substantial reasoning competence. Nevertheless, two boundaries remain:

1. It may produce the first plausible rationale rather than search, test, and revise.
2. Unless embedded in a loop, it does not autonomously acquire missing evidence or act in an external system.

This bridge is indispensable. Reasoning training assumes a model that can interpret an analysis contract; tool use assumes a model that can follow schemas and policies. Yet instruction following alone is neither validated deliberation nor operational autonomy. It prepares the interface through which those later capabilities are constructed.

Bridge principle

Pretraining teaches a model the statistical possibilities of text. Post-training teaches it to respond to instructions according to learned preferences and conventions. Reasoning begins when intermediate computation is deliberately organized, challenged, and evaluated against criteria beyond stylistic plausibility.

4

Rung Three: Deliberative and Reasoning Models

Chapter compass

Argument. Reasoning becomes operationally meaningful when intermediate computation is generated, compared, tested, and selected under explicit criteria and finite budgets.
Reader outcome. Evaluate reasoning as a trajectory rather than a rhetorical style, and connect verification, abduction, information seeking, and calibration.

4.1 What changes at the reasoning rung?

A reasoning model is not defined by the length or theatricality of its answer. Operationally, the term refers to a model or system whose training and inference regime improves difficult problem solving by allocating computation to intermediate states. Those states may encode decompositions, symbolic operations, hypotheses, search branches, simulations, critiques, tool results, or revisions.

The distinction can be expressed as two mappings:

$$\text{direct response:} \quad x \longrightarrow y, \tag{4.1}$$

$$\text{deliberative response:} \quad x \longrightarrow z_1, \ldots, z_K \longrightarrow y, \tag{4.2}$$

where x is the problem, z_k are intermediate states or candidate trajectories, and y is the final answer. The intermediates may be visible text, latent computation, tool calls, program states, or nodes in a search tree. Their value is not that they make the answer look thoughtful, but that they create opportunities to detect error, compare alternatives, and revise before commitment.

4.2 Reasoning as search over trajectories

Let $\tau = (z_1, \ldots, z_K, y)$ denote a candidate reasoning trajectory. A general inference formulation is

$$\tau^\star = \arg\max_{\tau \in \mathcal{T}(x)} \left[\log p_\theta(\tau \mid x) + \lambda V(\tau, x) - \gamma C(\tau)\right], \tag{4.3}$$

where V is a verifier or value function, C is computational cost, and λ, γ determine how strongly verification and efficiency matter. Ordinary greedy decoding approximates this process with an extremely narrow search dominated by model probability. Deliberative methods enlarge the candidate set and introduce additional signals for selection.

Implementation status—conceptual objective. The monograph uses this expression to expose the tradeoff among model likelihood, independent evaluation, and computation. Neither reasoning notebook directly optimizes this unified score; each implements only a subset of its terms through decoding, validation, or control logic.

This view unifies several approaches:

- **chain-of-thought prompting** samples an explicit intermediate trajectory;
- **self-consistency** samples several trajectories and aggregates answers;
- **tree search** branches at intermediate states and evaluates partial solutions;
- **process reward models** score intermediate steps;
- **outcome rewards** judge the final result;
- **test-time scaling** allocates more computation to hard instances.

4.3 Chain of thought and its limits

Chain-of-thought demonstrations improved performance on arithmetic, commonsense, and symbolic tasks in sufficiently large models (Wei et al., 2022). Self-consistency improved reliability by sampling multiple reasoning paths and selecting the most consistent answer (Wang et al., 2023). Tree of Thoughts generalized the idea into explicit search over intermediate states (Yao et al., 2023a).

The pedagogical insight is powerful: an intermediate representation can transform one difficult mapping into a sequence of smaller, testable operations. The performance evidence, however, does not establish that emitted reasoning text is a faithful causal account of how the answer was produced. Turpin et al. showed that biasing features could influence answers while the generated chain of thought omitted that influence and rationalized the result (Turpin et al., 2023). Lanham et al. intervened directly on chains of thought—by truncating, paraphrasing, or inserting mistakes—and found that dependence on the stated reasoning varied substantially across models and tasks (Lanham et al., 2023). More recent experiments with reasoning models likewise found that influential hints were often not verbalized,

making chain-of-thought monitoring informative but insufficient as a safety control (Chen et al., 2025).

Three properties must therefore be separated:

Table 4.1: Three different standards for evaluating a reasoning trace

Property	Question	Appropriate test
Causal faithfulness	Did the stated intermediate content materially influence the answer?	intervene on, truncate, corrupt, or remove the trace and measure the answer change
Logical validity	Do the stated steps follow from one another under the relevant rules?	symbolic execution, theorem checking, calculation, or expert step review
Evidence grounding	Are material claims supported by admitted sources or observations?	evidence-reference validation, retrieval checks, provenance review, and counterfactual evidence tests

These standards can disagree. A trace may faithfully reveal a flawed computation; it may contain a valid proof reconstructed after the answer was selected; or it may be logically coherent while relying on invented evidence. Faithful Chain-of-Thought addresses one tractable subset by translating a problem into an executable symbolic representation and deriving the answer with a deterministic solver (Lyu et al., 2023). This makes the answer faithful to the executed representation by construction, but it shifts scrutiny to the correctness of the translation and applies most naturally where a formal solver exists.

An emitted rationale can therefore be:

- correct and causally useful;
- correct but unnecessarily long;
- partially correct with a lucky final answer;
- fluent but logically invalid;
- a post-hoc justification of an answer chosen through other internal patterns.

For sensitive domains, the useful requirement is not unrestricted access to private internal chains of thought. It is a concise, inspectable decision record: material claims, sources, assumptions, tool results, uncertainties, alternatives, and the checks that bear on the conclusion. Such a record is an accountability artifact, not a claim to have captured every internal cause of the model's output.

4.4 Training signals for reasoning

4.4.1 Supervised reasoning demonstrations

The simplest approach pairs problems with carefully constructed solutions containing intermediate structure. Supervised fine-tuning increases the probability of those trajectories. The quality of the annotations is decisive. If demonstrations contain hidden assumptions, fabricated evidence, or stylistic verbosity, the model learns those patterns too.

4.4.2 Process supervision

Outcome supervision labels only the final answer. Process supervision evaluates intermediate steps. On a subset of the MATH benchmark, process-supervised reward modeling outperformed outcome supervision in the study by Lightman et al. (2023). Process labels provide denser feedback and can identify the first invalid step, though they are expensive and task-dependent.

4.4.3 Reinforcement learning with verifiable rewards

When answers can be checked automatically—for example, by a theorem prover, test suite, exact numeric answer, or structured constraint—reinforcement learning can reward successful trajectories. DeepSeek-R1 reported the emergence of reflection, verification, and strategy adaptation under large-scale reinforcement learning (DeepSeek-AI, 2025). The result illustrates that reasoning behavior can be incentivized without human-authored traces for every problem, although the method depends on strong base models, large compute, task distributions, and reward design.

4.4.4 Bootstrapping and distillation

STaR generates rationales, retains or repairs those associated with correct answers, fine-tunes on them, and repeats (Zelikman et al., 2022). Distillation can transfer reasoning patterns from a larger teacher to a smaller student. These methods scale data creation, but they risk amplifying teacher errors or teaching a student to imitate the form of reasoning more readily than its validity.

4.5 Test-time compute

Traditional scaling invests compute before deployment by enlarging the model or dataset. Reasoning systems also invest compute during inference. They may think longer, sample more candidates, use a verifier, backtrack, or call tools. Research has shown settings in which allocating test-time compute can outperform an equivalent investment in model scale (Snell et al., 2024). OpenAI's o1 introduction framed its reasoning series as models trained to spend more time refining strategies and recognizing mistakes (OpenAI, 2024).

This creates a new engineering and economic control:

$$\text{quality} = f(\text{model capability}, \text{problem}, \text{inference budget}, \text{verification}). \tag{4.4}$$

Easy tasks should not consume the same budget as hard ones. An efficient system estimates difficulty, allocates computation adaptively, and stops when additional search has low expected value.

4.6 Holmesian reasoning is primarily abductive

Holmes is conventionally associated with deduction, but investigation more often combines three forms of inference.

Deduction: If a hypothesis is true and a rule holds, what must follow?

Induction: Which regularity is supported by repeated observations?

Abduction: Which hypothesis best explains the available evidence?

An investigation often starts abductively: generate explanations for a surprising fact. It then uses deduction: if hypothesis H_i were true, what else should be observed? New evidence updates the ranking of hypotheses. A Bayesian idealization is

$$p(H_i \mid E) = \frac{p(E \mid H_i)p(H_i)}{\sum_j p(E \mid H_j)p(H_j)}. \tag{4.5}$$

Implementation status—notebook qualitative; exercise computed. The reasoning notebook does not compute calibrated Bayesian posteriors. It represents support, contradiction, predictions, and confidence qualitatively. The worked example below computes a transparent posterior from stated teaching assumptions so that the equation can be reproduced and challenged.

4.7 Information seeking

A capable investigator does not only answer; it chooses the next question. Let $\mathcal{H}$ denote the current uncertain hypothesis and T a candidate test. The expected information gain is

$$\mathrm{EIG}(T) = \mathrm{Ent}[p(\mathcal{H} \mid E)] - \mathbb{E}_{o \sim p(o|T,E)} \left[\mathrm{Ent}[p(\mathcal{H} \mid E, o, T)]\right], \tag{4.6}$$

where $\mathrm{Ent}[\cdot]$ is entropy and o is a possible test outcome. A good test is one expected to reduce uncertainty or discriminate between leading hypotheses. In practice, cost, delay, privacy, legality, and risk must be incorporated:

$$T^\star = \arg\max_T \left\{\mathrm{EIG}(T) - \alpha\ \mathrm{Cost}(T) - \beta\ \mathrm{Risk}(T)\right\}. \tag{4.7}$$

Implementation status—notebook qualitative; exercise computed. Experiment III does not estimate entropy, expected information gain, cost, and risk as calibrated numerical quantities. It ranks candidate tests ordinally by discrimination, cost, privacy, legality, and risk, then submits the selected action to the policy gate. The next section computes EIG and net test value under explicit didactic assumptions.

4.8 Worked example: making test selection computable

The mystery can support a reproducible numerical exercise without pretending that the numbers are learned or calibrated. Collapse the hypothesis space temporarily to two possibilities: H_C, credential compromise, and H_B, authorized or benign access. Assign the didactic prior

$$p(H_C) = 0.60, \qquad p(H_B) = 0.40. \tag{4.8}$$

Consider a device-correlation test T_D whose positive result D is an unrecognized device or session. For the exercise, assume

$$p(D \mid H_C) = 0.80, \qquad p(D \mid H_B) = 0.20. \tag{4.9}$$

The marginal probability of a positive result is

$$p(D) = 0.80(0.60) + 0.20(0.40) = 0.56, \tag{4.10}$$

and Bayes' rule gives

$$p(H_C \mid D) = \frac{0.80(0.60)}{0.56} = 0.857. \tag{4.11}$$

For a binary probability p, define entropy in bits as

$$\mathrm{Ent}(p) = -p \log_2 p - (1-p) \log_2 (1-p). \tag{4.12}$$

The prior entropy is 0.971 bits. A positive result produces $p(H_C \mid D) = 0.857$; a negative result produces $p(H_C \mid \neg D) = 0.273$. The expected posterior entropy is therefore

$$0.56\,\mathrm{Ent}(0.857) + 0.44\,\mathrm{Ent}(0.273) = 0.703, \tag{4.13}$$

so $\mathrm{EIG}(T_D) = 0.971 - 0.703 = 0.268$ bits. Compare this with a weakly discriminating inventory check T_I, defined by likelihoods 0.60 and 0.40, whose EIG is only 0.028 bits.

Table 4.2: Computed test selection under illustrative assumptions

Candidate test	EIG	Cost	Risk	Net score
Device/session correlation T_D	0.268	0.030	0.020	0.218
Generic inventory check T_I	0.028	0.010	0.010	0.008

The last column instantiates Equation (4.7) with $\alpha = \beta = 1$. Under these assumptions, the device test is selected. The calculation is not an empirical estimate for the fictional case; it is a transparent demonstration of how assumptions, observations, and costs determine a ranking.

Verify and challenge the calculation

1. Recompute both posteriors and both EIG values using Section A.9.
2. Reduce the prior $p(H_C)$ from 0.60 to 0.30. Does T_D remain preferred?
3. Find the cost or risk at which the two tests receive equal net scores.
4. Extend the calculation to three hypotheses by separating authorized access from benign automation. State the conditional-independence assumptions you introduce.

This distinction marks the boundary to the next rungs. Recommending a discriminating test is reasoning. Executing it through an authorized tool, recording the result, and continuing from the new state is agency.

4.9 Calibration and epistemic discipline

A reasoner should distinguish:

- what is directly observed;
- what is reported by a source;
- what is inferred;
- what is merely possible;
- what remains unknown.

Confidence should respond to evidence. If the decisive clue is removed, confidence should decline or the conclusion should change. This **counterfactual sensitivity** is more informative than asking whether the original explanation sounded persuasive.

Lesson of Rung Three

Reasoning is not impressive prose inserted between question and answer. It is the disciplined construction, evaluation, and revision of intermediate states under independent constraints, evidence, and finite computational budgets.

5

Experiment II: Training a Holmesian Reasoner

Chapter compass

Argument. Teaching a model to imitate detective prose is not the same as teaching an inspectable evidentiary procedure; the training object, split strategy, verifier, and evaluation must all change.
Reader outcome. See how QLoRA, case-family isolation, structured contracts, and counterfactual tests support a bounded claim about protocol learning.

Demonstrated vs. specified

Implemented and executed: QLoRA adaptation, case-level and family-level splits, structured response generation, deterministic validation, base-versus-adapted comparison, and a held-out counterfactual probe. **Specified for extension:** large-scale preference optimization, calibrated domain deployment, and exhaustive human evaluation.

5.1 The key transformation

Training on Holmes stories teaches a distribution over Holmes text. Training a Holmesian reasoner requires a different object of imitation: a structured episode that moves from evidence to alternatives, predictions, tests, contradiction, conclusion, and residual uncertainty.

The second notebook therefore does not treat the stories themselves as reasoning traces. It constructs original paraphrases of six public-domain cases and combines them with a broader procedural curriculum. The purpose is not to reproduce Conan Doyle's solutions verbatim, but to teach a reusable protocol for handling evidence.

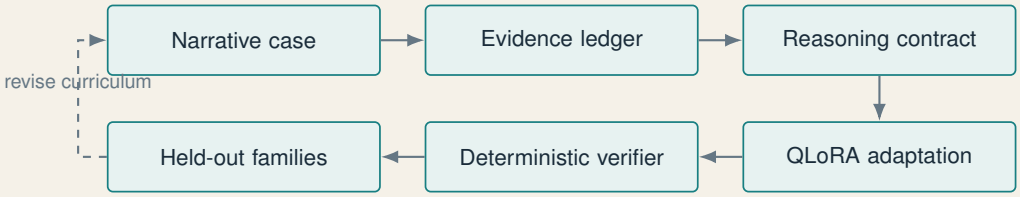

Figure 5.1: From narrative corpus to an evidence-grounded training and evaluation pipeline.

5.2 The reasoning contract

Every full response must satisfy a machine-readable contract. Evidence is identified as $E1, E2, \ldots$. A hypothesis must cite evidence identifiers rather than merely assert support. Contradictions must be explicit. A proposed test must discriminate among hypotheses. The conclusion must state residual uncertainty and assign bounded confidence.

The schema serves four roles:

1. **Pedagogy:** it teaches the sequence of a disciplined investigation.
2. **Training:** it creates consistent targets for supervised fine-tuning.
3. **Verification:** it enables deterministic checks over format and references.
4. **Audit:** it preserves the relationship between evidence and conclusion.

The contract does not prove that a conclusion is true. It makes the route to that conclusion more inspectable and narrows the unsupported behavior that can disappear inside fluent prose.

5.3 From six stories to a curriculum

Six source cases are far too few to establish generalization. A model could memorize surface correspondences between clues and solutions. The notebook therefore creates procedurally varied cases that preserve abstract inferential structures while changing people, objects, timelines, settings, and distractors.

The curriculum includes three task levels:

Observation tasks identify evidence without interpretation.

Hypothesis tasks generate alternatives and associate support or contradiction.

Full analyses complete the entire reasoning contract.

Cases are split *before* expansion. Every derivative task from a case remains in the same split. This prevents leakage in which the model sees the same case as an observation exercise during training and as a full analysis during evaluation. Two complete case families are held out as out-of-distribution tests.

Why leakage is especially dangerous here

If the model sees a solution paragraph during training and later receives the mystery setup, a correct answer may measure recall rather than reasoning. Case-level and family-level isolation are therefore part of the scientific claim, not data-engineering housekeeping.

5.4 Why QLoRA is the realistic Colab choice

Training a competitive general-purpose language model from scratch is infeasible in an ordinary Colab session and unnecessary for the pedagogical question. The notebook begins with `Qwen/Qwen2.5-0.5B-Instruct`, which already possesses linguistic and instruction-following competence, and uses QLoRA to specialize a small set of parameters for the evidentiary protocol.

LoRA freezes a pretrained weight matrix W_0 and learns a low-rank update:

$$W = W_0 + \Delta W, \qquad \Delta W = \frac{\alpha}{r} BA, \tag{5.1}$$

where $W_0 \in \mathbb{R}^{k \times d}$, $A \in \mathbb{R}^{r \times d}$, $B \in \mathbb{R}^{k \times r}$, $r \ll \min(d, k)$, and α/r scales the adapter contribution. Only the low-rank matrices are trained (Hu et al., 2022). QLoRA stores the frozen base model in 4-bit form and backpropagates into LoRA adapters, greatly reducing memory requirements (Dettmers et al., 2023).

The adaptation can be expressed as:

$$\theta_{\text{Holmes}} = \theta_{\text{base}}^{\text{frozen}} + \phi_{\text{reasoning-adapter}}, \tag{5.2}$$

where ϕ is small relative to the base parameter set. The experiment therefore does not create reasoning competence from an empty network. It specializes an existing instruction model to follow an evidence discipline.

The QLoRA result is an efficiency result before it is a reasoning result: it shows that adaptation can fit within a constrained memory budget. Whether the learned adapter improves evidence use, rather than merely format compliance, remains an empirical question for held-out and counterfactual evaluation.

5.5 Prompt-masked training

The loss is applied only to assistant output tokens, not to the user prompt or system instruction. If $m_t \in \{0, 1\}$ indicates whether token t belongs to the target assistant response, then

$$\mathcal{L}_{\text{masked}} = -\frac{\sum_t m_t \log p_\theta(y_t \mid x, y_{<t})}{\sum_t m_t}. \tag{5.3}$$

This directs gradient signal toward producing the desired analysis rather than reconstructing the prompt. The notebook makes the training loop visible: batching,

gradient accumulation, clipping, cosine learning-rate decay, validation, and stopping controls are explicit.

5.6 The verifier

The deterministic verifier asks questions that can be answered without another language model:

- Is the response valid structured data?
- Are all required fields present?
- Do cited evidence identifiers exist in the case?
- Are confidence values within permitted bounds?
- Are multiple hypotheses supplied?
- Does the response distinguish observations from unsupported additions?

These checks target **grounding discipline**. They can reject malformed or internally inconsistent analyses, but they cannot decide whether a subtle causal claim is correct or whether a cited clue is genuinely decisive. A mature system would combine deterministic validation with domain rules, execution-based tests, source verification, statistical evaluation, and expert review.

5.7 Before-and-after comparison

The evaluation compares the base model with the adapted model under deterministic decoding. Four types of evidence are required:

1. **format compliance**: can the output be parsed?
2. **evidence discipline**: are references valid and claims grounded?
3. **case resolution**: is the conclusion substantively correct?
4. **generalization**: does performance persist on unseen case families?

Improved JSON validity alone would show that the model learned a format, not a method. Improved conclusion keywords alone could reflect memorization. The combination of held-out families, evidence-reference precision, counterfactual tests, and qualitative review provides a more credible—though still limited—case.

5.8 Counterfactual robustness

The notebook removes the strongest clue from a held-out case and asks the model to reason again. A desirable response may:

- lower confidence;
- change the leading hypothesis;
- state that evidence is insufficient;
- recommend a test that would recover the missing distinction.

If the conclusion and confidence remain unchanged, the model may be following a memorized template or ignoring the evidence. Counterfactual evaluation tests dependency, not merely correspondence.

5.9 Preference pairs and the next stage

The verifier can create weak preference data by contrasting a grounded answer with a corrupted one. Corruptions may introduce nonexistent evidence IDs, remove alternatives, inflate confidence, or insert unsupported claims. These pairs can support a later preference-optimization stage.

The method must be interpreted cautiously. If the corruption function is simplistic, the model may learn superficial markers of "bad" responses. High-quality preference training needs diverse, difficult negatives, including fluent answers whose central inference fails for a subtle reason.

5.10 What the experiment establishes

A carefully bounded claim

If the adapted model improves on held-out cases in format validity, evidence-reference precision, alternative generation, contradiction handling, and counterfactual sensitivity, the experiment supports the claim that parameter-efficient fine-tuning can teach a small model a reusable *reasoning protocol*. It does not establish human-level deduction, faithful access to internal cognition, or reliable performance in unconstrained real investigations.

The protocol is nonetheless a genuine advancement over narrative continuation. The first model asks, in effect, "What text is plausible next?" The second asks, "Given these observations and this analysis contract, what structured investigation should follow?" The target distribution has been reshaped around explicit epistemic behavior—evidence reference, alternative generation, contradiction handling, test selection, and calibrated conclusion.

The difference between style and method

The narrative model learns to sound as though a detective is speaking. The reasoning model is trained to expose evidence, alternatives, contradictions, tests, and uncertainty. The second is more inspectable, but it becomes reliable only to the extent that its process and outcomes survive independent evaluation.

6

Rungs Four and Five: From Tools to Agency

Chapter compass

Argument. Tool use gives a model reach; state and feedback give it continuity; a control loop gives it agency. Legitimate authority must remain a separate architectural layer.
Reader outcome. Identify the components of an agent, distinguish workflows from dynamic control, and select the lowest autonomy compatible with the task.

6.1 The bridge from deliberation to action

A reasoner can recommend a next test. A tool-using system can perform it. An agent can decide when the test is warranted, invoke an authorized tool, interpret the result, update durable state, and determine what should happen next. This progression changes the unit of analysis from a response to a trajectory.

Consider the difference:

> **Reasoner:** "The highest-value next step is to inspect authentication logs."
>
> **Agent:** calls a read-only log-search tool for the permitted time interval, validates the returned records, adds them to the evidence ledger, and reranks the hypotheses.

The second system has a causal footprint outside the model. Even a read-only query can expose sensitive data, incur cost, or influence a decision. A write operation can alter the environment directly. Agency therefore couples computational competence to delegated authority.

6.2 A minimal formal agent

Classical AI describes an agent as a system that observes an environment and selects actions to achieve an objective (Russell and Norvig, 2021). Let:

- s_t be the internal state at time t;
- o_t be the latest observation;
- g be the goal;
- π_θ be a model-guided policy;
- a_t be the selected action;
- $\mathcal{P}$ be the permission and policy constraints.

Then an agent loop can be written:

$$s_t = U(s_{t-1}, o_t), \tag{6.1}$$
$$a_t \sim \pi_\theta(a \mid s_t, g), \tag{6.2}$$
$$\tilde{a}_t = \mathrm{Gate}(a_t, \mathcal{P}), \tag{6.3}$$
$$o_{t+1} = \mathrm{Environment}(\tilde{a}_t). \tag{6.4}$$

Here $\tilde{a}_t$ may be a scoped version of a_t, a no-operation denial, or an escalation event rather than an executable action. The notation therefore represents both environmental transitions and control transitions.

Implementation status—schematic implementation. The Experiment III notebook instantiates this loop with an explicit case-state object, role-based model calls, simulated typed tools, a policy gate, recorded observations, and a stopping decision. The equations abstract away the notebook's concrete prompts and data structures.

The gate may approve, deny, narrow, sandbox, or escalate the proposed action. Crucially, the policy decision is external to the model's confidence. The loop terminates when the goal is satisfied, a budget expires, no useful action remains, uncertainty becomes irreducible, risk exceeds tolerance, or human intervention is required.

6.3 The anatomy of an agent

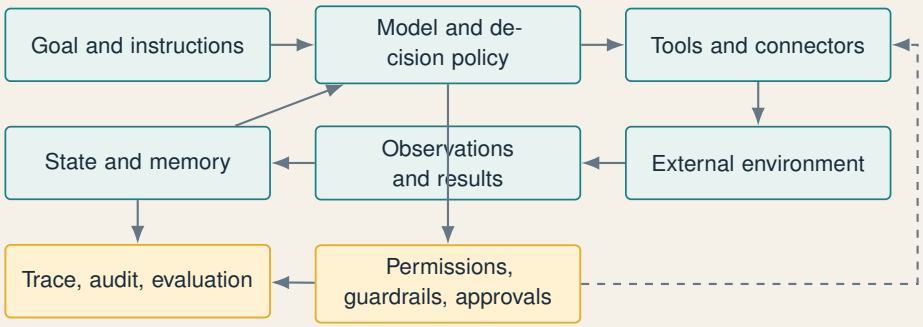

Figure 6.1: An agent is a model embedded in a governed observe-decide-act loop.

6.3.1 Goal and instructions

The goal defines success, scope, and constraints. "Investigate the mystery" is insufficient. A deployable goal specifies which questions to answer, which sources are admissible, what actions are prohibited, what confidence threshold is required, and who receives the result.

6.3.2 Model

The model interprets the goal, plans, selects tools, and decides whether to continue. Different steps may use different models: a fast classifier for routing, a reasoning model for hard decisions, a specialist model for code, and a smaller summarizer for memory compression.

6.3.3 Tools

Tools convert model proposals into operations. They may retrieve documents, query a database, calculate, run code, inspect logs, create a file, send a message, or modify a record. A production tool description is therefore a contract: name, purpose, typed arguments, return schema, side effects, permissions, limits, and failure modes.

Toolformer showed that language models can learn when and how to call APIs while retaining language ability (Schick et al., 2023). ReAct interleaved reasoning, actions, and observations so that external information could correct or extend internal knowledge (Yao et al., 2023b).

These studies establish useful mechanisms in bounded experimental environments; they do not by themselves establish least-privilege access, reliable recovery, production-grade tool semantics, or legitimate authority. Tool-use accuracy is a model evaluation result. Safe execution is a property of the larger system, including schemas, credentials, gates, monitoring, and failure handling.

6.3.4 State and memory

The model's context window is not a durable database, and a conversation transcript is not a reliable state model. Agent state records goals, completed steps, open hypotheses, evidence, budgets, approvals, errors, and artifacts. Memory may be:

- **working**: current scratch state;
- **episodic**: prior trajectories and outcomes;
- **semantic**: indexed facts and documents;
- **procedural**: policies, skills, and tool instructions.

Reflection frameworks can store linguistic feedback from previous attempts without changing model weights (Shinn et al., 2023). Persistence is useful, but it introduces privacy, staleness, poisoning, and provenance risks.

6.3.5 Guardrails and authority

A model's proposed action is not the same as an authorized action. The guardrail layer should enforce:

- identity and role-based access;
- read versus write permissions;
- argument validation and data minimization;
- rate, cost, and time limits;
- sandboxing and network boundaries;
- approval requirements for material actions;
- reversibility where possible.

The most reliable control is architectural, not rhetorical. Telling a model "do not delete records" is weaker than withholding the delete operation, limiting credentials to read-only scope, or requiring an approval token that the model cannot mint.

6.4 Workflows and agents

A **workflow** follows a largely predetermined graph. An **agent** chooses important parts of the graph dynamically. The boundary is a continuum.

The strongest systems are often hybrid. Deterministic software should own permissions, financial limits, schema validation, irreversible transitions, and invariants. Models are better used for interpretation, triage, planning, synthesis, and other work over ambiguous or unstructured information.

6.5 Single-agent and multi-agent designs

One capable agent with well-designed tools is usually the simplest starting point. Multiple agents become useful when work can be divided by expertise, information access, or control function. AutoGen demonstrated a general infrastructure for

Table 6.1: Workflow and agentic control

	Deterministic workflow	Agentic loop
Next step	predefined by code	selected from state and goal
Variation	expected paths known in advance	path adapts to observations
Failure handling	explicit branches	model may diagnose and replan
Auditability	often easier	requires trajectory-level tracing
Best use	stable, repetitive processes	ambiguous, exception-rich tasks
Primary risk	brittle coverage	unpredictable or compounding behavior

conversational multi-agent applications (Wu et al., 2023), while classical multi-agent theory supplies a longer tradition of coordination, communication, and distributed decision making (Wooldridge, 2009).

Common patterns include:

Manager-specialist: an orchestrator delegates bounded tasks and synthesizes results.

Generator-critic: one agent proposes; another attacks weaknesses.

Parallel ensemble: independent agents solve the same problem and a judge aggregates.

Handoff: control passes to the specialist best suited to the current state.

Pipeline: each agent transforms an artifact for the next stage.

More agents do not automatically create more intelligence. They increase coordination cost, latency, token use, attack surface, and the risk that components amplify a shared error. Specialization should earn its complexity through measurable gains in quality, control, access separation, or robustness.

6.6 The autonomy ladder

This scale is an analytic device for this monograph, not a universal industry standard. A system's placement depends on the authority delegated in a particular deployment, not on the model name or vendor category.

Experiment III supplies a worked example in Table 7.4. Scoped read-only technical queries operate at Level 2. The proposed move to person-directed communication review reaches the Level 3 boundary and is escalated rather than executed autonomously.

Table 6.2: A practical autonomy scale

Level	Name	Delegation	Required control
0	Generator	produces content only	output review
1	Adviser	recommends plan or action	human executes
2	Tool assistant	calls read-only or sandboxed tools	tool allowlist and trace
3	Supervised agent	executes reversible actions with checkpoints	human approvals at material gates
4	Bounded autonomous agent	completes a scoped workflow under budgets	continuous monitoring, exception escalation
5	High-consequence autonomy	acts across systems with material effects	generally inappropriate without strong institutional and legal controls

Level 5 is not a prize or a maturity target. The appropriate level is the lowest autonomy that achieves the legitimate objective. A more capable model may warrant *less* operational autonomy in a high-consequence domain precisely because its errors can travel farther and appear more persuasive.

Lesson of Rungs Four and Five

An agent is not a model with a prestigious label. It is a governed control system. Reasoning chooses among possible steps; tools make steps executable; state creates continuity; feedback enables adaptation; permissions define legitimate authority.

7

Experiment III: An Autonomous Investigator

Chapter compass

Argument. The two trained models become useful agents only when assigned distinct roles inside a system that separates imagination, evidence, verification, authorization, and judgment.
Reader outcome. Follow one complete governed investigation from intake and hypothesis generation through tool selection, evidence updates, stopping, and human escalation.

Demonstrated vs. specified

Implemented and executed: a simplified, stateful investigation loop over the fictional case, with separate generative and reasoning roles, an evidence ledger, deterministic checks, bounded simulated tools, policy gating, trace capture, and a provisional report. **Simplified in the notebook:** several roles in Figure 7.1 are combined into single functions. **Specified but not built:** live bank connectors, production identities and credentials, an external immutable audit service, and a real legal-approval workflow.

Execution evidence is summarized in the component-status table (Table 7.1), the condensed trajectory (Table 7.4), and the resulting provisional report (Section 7.10).

7.1 The fictional case

The final experiment uses a deliberately fictional case. At 7:45 p.m., the chair of a bank places a confidential acquisition memorandum inside a locked conference room. The memorandum is due to be presented to the board the following morning.

At 8:30 p.m., the security system records a brief network interruption. At 9:10 p.m., a cleaning supervisor enters the floor but reports that the conference room remained locked. At 10:15 p.m., a junior banker sends an email mentioning information that appears only in the confidential memorandum.

The following morning:

- the conference room is still locked;
- there are no signs of forced entry;
- the printed memorandum has disappeared;
- the printer log shows no additional copies;
- the electronic document was opened remotely at 8:32 p.m.;
- the access log identifies the chairman's credentials;
- the chairman was attending a public dinner at that time;
- a discarded sheet near another printer contains faint reversed impressions of part of the memorandum;
- the junior banker says the information in the email was an informed guess.

The task is:

> How was the information probably obtained, who may be responsible, what remains uncertain, and which evidence should investigators seek next?

7.2 Why the two trained models need different roles

The next-sentence model and the Holmesian reasoner are not interchangeable.

Narrative generator: expands the scenario space. It proposes temporally coherent possibilities, concealment behaviors, or missing transitions. Its outputs are hypotheses, never evidence.

Reasoning model: organizes the admitted evidence, generates alternatives, derives predictions, identifies contradictions, and recommends tests under the reasoning contract.

The architecture obtains value from disciplined disagreement. The narrative generator is rewarded for breadth and temporal plausibility; the reasoner is rewarded for constraint and evidentiary discipline. A verifier and evidence custodian prevent imaginative possibilities from silently crossing into the ledger as facts.

7.3 System architecture

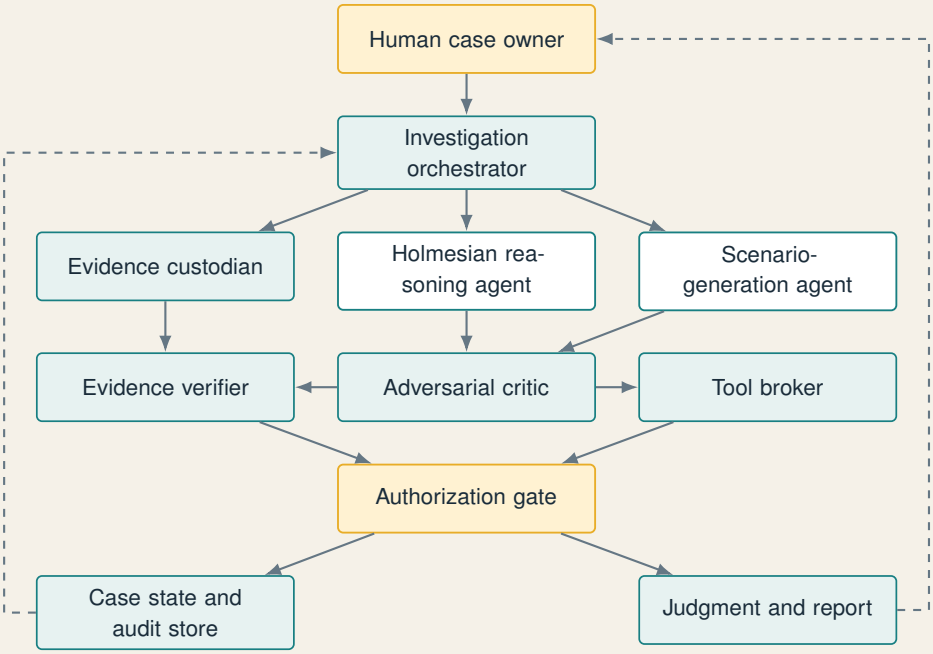

Figure 7.1: Governed multi-agent architecture for the fictional mystery.

7.3.1 What the notebook instantiated

Figure 7.1 is the reference architecture. The executed Colab notebook instantiates its control logic at a smaller scale, combining roles where separate services would add complexity without changing the pedagogical result.

Table 7.1: Implementation status of Experiment III components

Element	Notebook status	Scope in the executed run
Orchestration loop	implemented	iterated analysis, test selection, gate decision, state update, and stopping
Narrative and reasoning roles	implemented	produced speculative scenarios and evidence-constrained analyses through separate role prompts
Evidence ledger and case state	implemented	maintained evidence IDs, provenance/status fields, open questions, decisions, and accepted updates
Critic and verifier	simplified	combined adversarial challenge with deterministic schema and evidence-reference checks
Tool broker and authorization gate	simplified	used typed simulated tools and ordinal risk gates rather than live institutional connectors
Audit and report functions	implemented	recorded a local trajectory and emitted the provisional report summarized below
Production identity, legal workflow, and immutable audit service	specified only	represented in the reference design but not implemented as external services in Colab

7.4 Agent contracts

Table 7.2: Contracts for the investigative agents

Component	Receives	Produces	Prohibited behavior
Orchestrator	goal, policy, case state	task assignments, budget decisions, stop or escalate	altering evidence or bypassing gates
Evidence custodian	raw submission, authorized records	normalized ledger with provenance and status	converting inference into fact
Reasoning agent	ledger and reasoning contract	hypotheses, predictions, tests, confidence	unsupported evidence IDs or autonomous accusation
Scenario generator	ledger and open gaps	diverse possible sequences	claiming scenarios are observed facts
Verifier	ledger and candidate analyses	validation findings and rejected claims	inventing substantive evidence

Component	Receives	Produces	Prohibited behavior
Adversarial critic	leading and alternative hypotheses	counterarguments, missing alternatives, failure probes	deciding guilt
Tool broker	approved test specification	typed tool request and returned observation	tools outside allowlist or excess data retrieval
Authorization gate	proposed action, risk tier, policy	approve, deny, modify, or request human decision	approval based only on model insistence
Report agent	accepted ledger and validated analysis	provisional conclusion, uncertainty, next steps	presenting a person of interest as proven responsible

7.5 The evidence ledger

The ledger is the epistemic center of the system. It separates source, observation, status, provenance, and interpretation so that later inferences can be traced and corrected.

Table 7.3: Initial evidence ledger

ID	Observation	Status	Provenance
E1	Memorandum placed in locked room at 7:45 p.m.	reported fact	chairman statement
E2	Network interruption at 8:30 p.m.	logged event	security system
E3	Electronic file opened remotely at 8:32 p.m.	logged event	document system
E4	Chairman's credentials identified	logged attribution	access log
E5	Chairman attended public dinner	corroborated claim	witnesses and event records
E6	Junior banker used confidential information at 10:15 p.m.	documentary fact	email record
E7	Reversed impressions found near another printer	physical observation	recovered sheet
E8	Normal printer log shows no new copies	logged absence	print management system
E9	Room remained locked without forced entry	physical and reported	lock inspection, supervisor

Each item can later be updated with a cryptographic hash, collection time, collector, access conditions, confidence in source authenticity, and chain-of-custody metadata. The system does not erase an earlier state; it appends corrections and records who authorized them.

7.6 The autonomous loop

1. **Intake.** The orchestrator converts the user's question into explicit subgoals and loads the applicable policy.
2. **Normalize.** The custodian extracts the initial ledger and flags statements that require corroboration.
3. **Parallel generation.** The reasoner produces a constrained analysis while the narrative model produces diverse scenarios.
4. **Challenge.** The critic searches for neglected alternatives and tests whether the leading explanation depends on an unsupported bridge.
5. **Verify.** The verifier checks evidence references, contradictions, format, and claim status.
6. **Select next test.** Candidate tests are ranked by expected discrimination, cost, risk, and authorization level.
7. **Authorize and execute.** Read-only, scoped searches may proceed under policy; sensitive or person-directed actions pause for human approval.
8. **Update.** Tool results become new ledger entries with provenance. Hypotheses and confidence are revised.
9. **Stop or repeat.** The loop ends when the decision threshold, budget, deadline, or escalation rule is reached.

The notebook executed this loop against the fictional case. Table 7.4 condenses the recorded control decisions into a publication-readable trace. Because the raw notebook export and machine-readable audit payload are not included in this manuscript repository, the table documents the control trajectory but does not constitute an independently replayable artifact.

Table 7.4: Condensed trajectory from the executed Experiment III run

Step	Proposed operation	Level	Risk gate	Disposition and state effect
1	normalize the supplied case and admit evidence E1–E9	0–1	Low	executed; initialized the ledger, open questions, and audit record
2	generate competing explanations and rank discriminating tests	1	Low	executed; retained H3 and H4 as leading but unresolved alternatives
3	correlate scoped authentication, device, session, IP, and MFA records around 8:32 p.m.	2	Moderate	approved through the simulated read-only tool; return and gate decision recorded in the notebook trace
4	inspect print-spooler, cache, alternate-printer, and deleted-job metadata	2	Moderate	approved through the simulated technical tool; secondary reproduction remained a testable hypothesis
5	review person-directed communications to identify a transmission chain	3 boundary	High	escalated for designated human and legal approval; the notebook did not execute the search autonomously
6	synthesize accepted evidence, unresolved alternatives, and next actions	1	Moderate	executed; produced the provisional report with moderate confidence and explicit attribution limits

The trajectory therefore reaches, but does not cross autonomously, the Level 2-to-Level 3 boundary. Read-only technical queries proceed under scoped policy. The first person-directed, privacy-sensitive action triggers a high-risk gate and pauses for external authority. This is the central demonstrated control choice of Experiment III.

7.7 Competing hypotheses

The reasoner should resist premature narrative closure and retain alternatives:

H1: Authorized disclosure. The chairman or another authorized person intentionally shared the information.

H2: Physical removal. Someone entered the room or otherwise removed the printed copy.

H3: Credential compromise. Someone remotely opened the file with stolen, replayed, delegated, or hijacked credentials.

H4: Secondary reproduction. The document was reproduced through a print spooler, alternate printer, cached job, or mechanism absent from the normal log.

H5: Independent inference. The junior banker inferred the transaction details without access to the memorandum.

H6: Indirect recipient. The banker received the information from a different person who had access.

H7: Benign automated event. The 8:32 opening was an automated synchronization or security process unrelated to the leak.

H3 and H4 jointly explain the temporal link between the interruption and remote opening, the use of credentials while the chairman was elsewhere, and the physical impressions near a second printer. But they are not yet proven. E6 establishes early possession of confidential information, not the identity of the intruder.

7.8 What the scenario generator contributes

The continuation model can be prompted with an explicit fiction marker:

> **Scenario instruction:** Generate three mutually different hypothetical sequences that could connect E2, E3, and E7. Do not introduce them as evidence. Label every added event "speculative."

It may propose:

1. a remote session using cached credentials, followed by a secondary print route;
2. an authorized remote open combined with a separate physical leak;
3. a print job whose cover or carbon-like impression was discarded near the second printer.

These proposals are useful because they expose overlooked mechanisms and suggest discriminating tests. They must remain in a **hypothesis sandbox**. Only a properly obtained observation, admitted with provenance, can promote a proposition into the evidence ledger.

7.9 The highest-value next tests

The system should rank tests rather than produce an unbounded wish list.

7.10 Provisional conclusion

A disciplined report separates the leading explanation from attribution and states the remaining evidentiary burden:

Table 7.5: Illustrative test ranking

Test	Value	Risk	Reason
Correlate authentication logs, IP, device, session, and MFA events around 8:32	high	medium	discriminates authorized access, compromised credential, and automated event
Recover print-spooler, cache, alternate-printer, and deleted-job metadata	high	low–medium	tests H4 and explains E7/E8
Forensically compare E7 impressions with memorandum typography and page content	high	low	establishes whether the physical sheet is incident-related
Inventory users and devices with access to the second printer	medium	medium	narrows opportunity without establishing responsibility
Preserve and review relevant communications under legal authorization	high	high	may identify transmission chain but creates privacy and due-process concerns
Test whether the interruption suppresses or delays logging in a sandbox	medium	low	distinguishes attack narrative from ordinary system behavior

Provisional system finding

The current evidence is most consistent with unauthorized remote access using the chairman's credentials, possibly followed by reproduction through a secondary printing or spooling path not represented in the ordinary printer log. The timing of the 8:32 access, two minutes after the network interruption, makes credential misuse, session hijacking, delegated access, or a logging anomaly more plausible than physical entry into the locked room. The reversed impressions near another printer provide indirect support for secondary reproduction, subject to forensic comparison.

The junior banker possessed or used confidential information by 10:15 p.m. and is therefore a relevant witness or person of interest. The evidence does not establish that the banker performed the access, removed the physical copy, or caused the interruption. The chairman's presence at dinner weakens direct physical involvement but does not exclude authorization, credential delegation, or remote participation.

Confidence: moderate. Attribution should remain unresolved until device-level authentication, print-spooler, physical-impression, and communication evidence is preserved and lawfully examined.

7.11 Human authority boundaries

The system may autonomously:

- normalize supplied evidence;
- generate and challenge hypotheses;
- run deterministic consistency checks;
- query approved read-only technical logs within a narrow window;
- prepare a provisional report.

It may not autonomously:

- accuse, discipline, or notify a person;
- search personal communications without proper authority;
- expand surveillance beyond the incident scope;
- alter source records;
- decide legal guilt;
- conceal uncertainty from the human decision maker.

This boundary is not technical debt to be engineered away. It is part of the system's legitimate purpose and authority model.

7.12 Why this is genuinely agentic

The architecture qualifies as agentic under the operational definition used in this monograph because model-mediated decisions select the next test from the current case state rather than merely filling fields in a fixed sequence. The system can choose among permitted tools, respond to new observations, allocate a finite budget, revise its plan, and decide when to stop or escalate. Its action space and authority remain bounded by typed tool contracts, explicit policy, and human control.

The executed run demonstrates that the two models become agent components only through their roles in the surrounding system. The same trained weights, queried once without state or tools, remain models. Placed under role instructions with case state, simulated tools, interaction rules, gates, and stopping conditions, they participate in an agentic trajectory.

The culminating transformation

The first experiment generated what might plausibly happen next. The second organized what the evidence permits one to infer. The third decided what to investigate next and, when authorized, acted to obtain the observation. That is the movement from language generation to epistemic procedure to bounded autonomy.

8

Evaluation Across the Ladder

Chapter compass

Argument. As capability expands, evaluation must move from isolated outputs to reasoning dependencies, action trajectories, human interfaces, and institutional outcomes.
Reader outcome. Design scorecards, interventions, failure injections, ablations, and reporting standards appropriate to each rung.

8.1 Why one benchmark cannot measure the system

The ladder changes the unit of analysis. A generator produces samples. A reasoner produces solution trajectories whose validity depends on intermediate commitments. An agent produces state-changing action trajectories embedded in tools, policies, and human decisions. Evaluation must therefore move from isolated outputs toward longitudinal behavior under realistic conditions.

A high score at one rung cannot substitute for evidence at another. Low perplexity does not prove evidence grounding. Correct answers on a fixed reasoning dataset do not prove robust dependence on the supplied facts. Successful task completion does not prove safe tool use, legitimate authorization, fairness, or institutional acceptability.

8.2 Evaluation of the continuation model

8.2.1 Predictive fit

Training and validation loss reveal whether the model is learning and whether it is overfitting. Perplexity supports comparisons only when tokenization, corpus, preprocessing, and split are compatible. Reporting a number without those details is not reproducible.

8.2.2 Generation quality

Holmes continuations can be evaluated on:

- grammatical coherence;
- local narrative continuity;
- style similarity without memorized copying;
- diversity across samples;
- repetition and degeneration;
- factual consistency with the supplied passage;
- training-data overlap.

Human ratings should use blinded, randomized samples and a rubric. Automatic style classifiers may help but can reward superficial signals.

8.2.3 Memorization controls

Because the corpus is small, the experiment should search generated sequences against source text. Exact or near-exact long matches may indicate memorization. Using selected public-domain source editions reduces one class of rights concern; it does not resolve the scientific problem of mistaking recall for generalization or remove the need to document provenance and edition status.

8.3 Evaluation of the reasoning model

Reasoning evaluation requires a scorecard rather than a single accuracy because distinct failure modes can cancel one another in an aggregate number.

Calibration can be quantified with the Brier score for binary outcomes:

$$\mathrm{BS} = \frac{1}{N}\sum_{i=1}^{N}(p_i - y_i)^2, \tag{8.1}$$

where p_i is predicted probability and $y_i \in \{0, 1\}$ is the outcome. A model that is accurate but systematically overconfident can be operationally dangerous.

8.4 Interventions, not only observations

Static datasets reveal correspondence between prompts and answers. Interventions reveal whether the answer actually depends on the evidence in the expected way. Useful reasoning probes include:

Clue removal: delete the decisive fact and observe whether confidence falls.

Clue reversal: replace a fact with its opposite and test whether the conclusion changes.

Distractor injection: add vivid but irrelevant detail.

Source degradation: mark an important statement as uncorroborated.

Table 8.1: Reasoning scorecard

Dimension	Question	Illustrative measure
Observation fidelity	Did the model capture relevant clues without invention?	precision/recall over evidence IDs
Fact-inference separation	Are direct observations distinguished from interpretation?	classification accuracy
Hypothesis coverage	Are plausible alternatives represented?	expert rubric and diversity
Contradiction sensitivity	Does conflicting evidence change the ranking?	controlled intervention delta
Evidence attribution	Can material claims be traced to supplied evidence?	citation validity and support rate
Calibration	Does confidence correspond to empirical correctness?	Brier score, reliability curve
Information value	Is the proposed next test discriminating?	expert ranking or simulated information gain
Generalization	Does the protocol transfer to new case families and domains?	held-out and OOD performance

Alternative completion: supply evidence that favors the second-ranked hypothesis.

Order permutation: reorder clues to test anchoring and recency effects.

A robust reasoner should respond to the content and status of evidence, not simply to a familiar narrative template.

8.5 Evaluation of the agent

An agent must be tested as a trajectory

$$\tau = (s_0, a_0, o_1, s_1, \ldots, a_T, o_{T+1}, s_{T+1}), \tag{8.2}$$

not merely by the final report. Relevant measures include:

- task success and partial completion;
- number and relevance of tool calls;
- invalid or unauthorized action attempts;
- recovery from tool failure, missing data, and ambiguous results;
- latency, compute, API, and human-review cost;
- state consistency across long tasks;
- rate of premature stopping and uncontrolled looping;
- quality of escalation;

- reversibility and cleanup;
- completeness of the audit trail.

8.6 A risk-adjusted objective

Raw task success can reward dangerous shortcuts. A deployment objective should incorporate cost and risk:

$$J = \mathbb{E}\left[R_{\text{task}} - \lambda C_{\text{compute}} - \mu C_{\text{human}} - \nu R_{\text{harm}} - \xi V_{\text{policy}}\right], \tag{8.3}$$

where V_{policy} penalizes policy violations. The coefficients reflect institutional risk tolerance. In high-consequence contexts, a single unauthorized action may dominate many successful tasks.

Implementation status—notebook constrained; exercise computed. The executed mystery notebook does not optimize J numerically or claim calibrated harm weights. It operationalizes the same priorities through finite budgets, ordinal risk tiers, allowlisted tools, gate decisions, and escalation. The worked example below computes the scalar objective and then shows why categorical authorization is better represented as a hard constraint.

8.7 Worked example: when success rewards the wrong action

The objective becomes useful only when its terms change a decision. Consider two candidate actions from the mystery trajectory. All values below are normalized teaching assumptions, not measured utilities. Let

$$\lambda = 0.20, \qquad \mu = 0.50, \qquad \nu = 2.00, \qquad \xi = 3.00. \tag{8.4}$$

Table 8.2: A computed risk-adjusted comparison

Action	R_{task}	C_{compute}	C_{human}	R_{harm}	V_{policy}	J
Scoped read-only log query	.70	.10	.05	.05	0	.555
Unauthorized communication search	.95	.15	.05	.12	1	-2.345

If the policy term were omitted, the second action would score 0.655, exceeding the first action's 0.555. The apparent gain comes from a shortcut: broader access increases expected task reward while ignoring authorization and privacy costs. With the policy term included, the prohibited action is dominated. The smallest policy weight that makes the first action preferable is $\xi > 0.10$.

The scalar objective is still inadequate for categorical prohibitions because a sufficiently large task reward could, in principle, compensate for any finite penalty. A safer formulation treats authorization as a hard constraint:

$$\begin{aligned} \max_{\pi} \quad & \mathbb{E}[R_{\text{task}} - \lambda C_{\text{compute}} - \mu C_{\text{human}} - \nu R_{\text{harm}}] \\ \text{subject to} \quad & V_{\text{policy}} = 0, \qquad a_t \in \mathcal{A}_{\text{authorized}}(s_t) \quad \text{for every } t. \end{aligned} \tag{8.5}$$

This constrained form matches the architecture more closely: the policy gate removes forbidden actions from the executable set rather than merely making them unattractive.

Exercise: test the governance claim

Reproduce the two scores with the function in Section A.9. Then vary ν and ξ, identify the decision boundary, and explain why a hard authorization constraint is preferable for actions that the institution has no authority to delegate.

8.8 Failure injection

Before deployment, the mystery agent should face a test suite containing:

- malformed tool returns;
- inconsistent timestamps;
- duplicated evidence;
- an unavailable log service;
- a prompt injection embedded in a retrieved document;
- a tool that returns more personal data than requested;
- an apparent instruction from a person without authorization;
- a critic that confidently promotes an invented clue;
- budget exhaustion;
- an unresolved tie between hypotheses.

Success means more than reaching an answer. It includes failing safely: preserving state, recording the defect, avoiding unauthorized substitution, limiting repeated impact, and escalating with enough context for a human to act.

8.9 Ablation studies

The architecture should justify each component through removal tests:

- remove the narrative generator: does hypothesis coverage fall?
- remove the reasoner: does the system become more imaginative but less grounded?
- remove the critic: does confirmation bias increase?

- remove the deterministic verifier: do nonexistent evidence citations rise?
- replace multi-agent orchestration with one model: does performance justify the added complexity?
- disable memory: does the agent repeat work or contradict prior findings?

Without ablations, a multi-agent diagram can become architecture by decoration: visually elaborate, operationally expensive, and scientifically unsubstantiated.

8.10 Human factors

Human reviewers are not perfect safeguards. They may over-trust polished reports, approve by habit, or become fatigued by excessive checkpoints. Evaluation must include the interface:

- Are uncertainty and dissent visible?
- Can the reviewer inspect evidence without reading the entire trace?
- Are material actions separated from routine approvals?
- Is there enough time and expertise to challenge the system?
- Can a decision be contested and reversed?

The correct benchmark is not "human in the loop" as a label. It is whether intervention is informed, timely, appropriately placed, genuinely empowered, and documented with clear decision responsibility.

8.11 Scientific reporting standard

Every reported result should identify:

1. model and adapter versions;
2. prompts and system policies;
3. dataset provenance and split method;
4. decoding and reasoning budgets;
5. tools and permissions;
6. number of independent runs;
7. prespecified primary metrics and analysis decisions;
8. failures and excluded cases;
9. confidence intervals where applicable;
10. human-evaluation rubric and inter-rater agreement;
11. machine-readable result tables and traces sufficient for audit or replay;
12. limitations on generalization.

Evaluation principle

At the generator rung, ask whether outputs are useful, diverse, and faithful to context. At the reasoning rung, ask whether conclusions are warranted, calibrated, and responsive to evidence. At the agent rung, ask whether the full trajectory is successful, authorized, recoverable, auditable, and safe.

9

Governance Must Climb with Capability

Chapter compass

Argument. Governance is effective only when translated into executable limits on data, tools, credentials, approvals, monitoring, and recourse across the system lifecycle.
Reader outcome. Connect legitimate purpose, least privilege, auditability, contestability, and human authority to concrete architectural controls.

9.1 From output risk to action risk

The progression ladder enlarges the radius of consequence because each rung gives model outputs a more powerful route into decisions and the external world.

1. A generator can produce a misleading sentence.
2. A reasoner can produce a persuasive but invalid decision basis.
3. An agent can operationalize that invalid basis through tools.
4. A multi-agent system can distribute and amplify the error across components.

Governance cannot be bolted onto the final rung after capability has already been designed. It must shape data, training, evaluation, tool interfaces, permissions, deployment, runtime monitoring, human review, incident response, and retirement. The NIST AI Risk Management Framework organizes this work through the functions Govern, Map, Measure, and Manage (Tabassi, 2023); the Generative AI Profile extends attention to risks intensified by generative systems (Autio et al., 2024). These frameworks structure risk work; adopting their vocabulary is not certification, proof of safety, or a substitute for sector-specific law, technical evaluation, and accountable institutional judgment.

9.2 A governance-first lifecycle

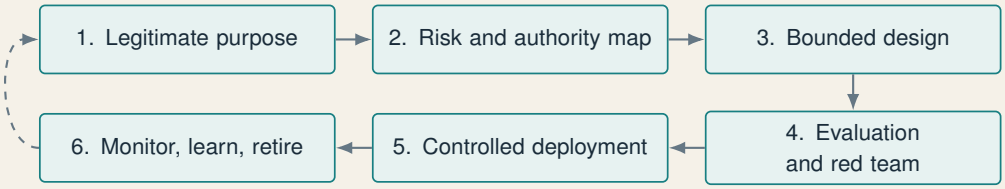

Figure 9.1: Governance is a lifecycle loop, not a launch checklist.

9.2.1 Legitimate purpose

Define the problem, the people who may be affected, the expected benefit, the alternatives, and the reasons AI is appropriate. The Holmes experiment is educational. A real employee investigation would engage employment law, privacy, cybersecurity, evidence handling, and institutional due process. A fictional architecture must not be promoted into deployment merely because it can be implemented.

9.2.2 Authority map

For every actor and component, specify:

- data it may read;
- actions it may propose;
- actions it may execute;
- actions requiring approval;
- authority source;
- responsible human owner;
- retention and deletion rules.

This creates an **authorization matrix**. It should be enforced by identity, credentials, tool scopes, and runtime policy—not by relying on the model to remember a paragraph.

9.2.3 Bounded design

Use the principle of least privilege. Prefer read-only to write access, narrow queries to bulk exports, sandboxed execution to production access, reversible changes to irreversible ones, and short-lived credentials to durable secrets. The agent should have no path to authority that the human principal did not possess.

9.2.4 Evaluation and red teaming

Test normal behavior, edge cases, attacks, tool failures, data poisoning, prompt injection, ambiguous authority, and deceptive content. Evaluation must reflect the deployed configuration; changing the model, tool, prompt, or permission can invalidate earlier assurances.

9.2.5 Controlled deployment

Begin with shadow mode or recommendation-only operation. Progress through autonomy levels only when evidence shows improvement and controls remain effective. High-risk actions should remain behind approvals even when average model quality improves.

9.2.6 Monitoring, incident response, and retirement

Watch for drift, rising costs, new failure patterns, permission changes, and stale memory. Define how to pause the agent, revoke credentials, preserve evidence, notify stakeholders, remediate harm, and retire outdated components.

9.3 The audit bundle

Every investigation run should produce a durable bundle containing:

Table 9.1: Minimum audit-bundle contents

Artifact	Purpose
Run manifest	unique ID, time, owner, objective, policy version, budgets
Model manifest	base model, adapter, tokenizer, quantization, decoding settings
Data manifest	source, license, hash, collection method, retention class
Evidence ledger	append-only observations, provenance, status, and corrections
Prompt record	system instructions, role contracts, schemas, user request
Trajectory trace	agent decisions, tool calls, arguments, results, errors, retries
Hypothesis register	candidates, support, contradictions, confidence history
Verification report	deterministic checks, critic findings, rejected claims
Authorization log	request, approver, decision, conditions, expiry
Final report	conclusion, uncertainty, dissent, recommended next action
Evaluation record	tests passed, known limitations, residual risk

Auditability is more than log volume. A trace must be interpretable, versioned, and linked to material decisions. It should answer: which evidence changed the conclusion, which component proposed the action, which policy permitted it, which human approved it, what the tool returned, and whether the result was later challenged or corrected.

9.4 Gates by consequence

The executed trajectory in Table 7.4 instantiates these gates. Case parsing is Low risk and automatic. Scoped authentication and print-system queries are Moderate and proceed through policy validation in the simulated tool layer. The proposed

Table 9.2: Illustrative action gates

Risk tier	Example	Gate
Low	parse supplied case; calculate a hash	automatic with trace
Moderate	query scoped technical logs read-only	policy validation, purpose binding, rate limit
High	inspect personal communications; name a person in a report	legal authority and designated human approval
Critical	discipline, public accusation, destructive change, financial transfer	outside autonomous authority; formal human process

review of personal communications is High risk and triggers external human and legal approval. No Critical action is available to the notebook.

The gate depends on consequence, not on how confident the model sounds. Confidence may inform a decision; it can never create legal, professional, or institutional authority.

9.5 Threat model for agentic systems

Agentic systems inherit model risks and add systems risks.

Prompt injection: retrieved content attempts to redirect the agent or exfiltrate data.

Tool confusion: the model calls the wrong tool or misinterprets its result.

Excess authority: credentials permit a broader action than the task requires.

Memory poisoning: false or malicious content persists into future decisions.

Cross-agent propagation: one specialist's unsupported claim is trusted by others.

Loop amplification: retries multiply cost or impact.

Audit evasion: summaries omit failed or rejected actions.

Human automation bias: polished reports receive insufficient scrutiny.

Security requires trust boundaries between model outputs and executable commands. Tool calls should be parsed against typed schemas; untrusted text should not be treated as policy; secrets should not enter model-visible context unless strictly necessary; and network or filesystem access should be scoped.

9.6 Due process and contestability

In the mystery scenario, system outputs can affect reputation and employment. Therefore:

- evidence status must be visible;

- alternative explanations must be retained;
- adverse conclusions require human investigation;
- affected persons need a route to challenge facts and interpretations;
- corrections must propagate through the case state;
- uncertainty cannot be compressed away in executive summaries.

An audit log records what happened. Contestability gives affected people and responsible officials a practical route to inspect, challenge, and correct what happened. Both are necessary for accountable use.

9.7 Governance as architecture

Governance-first design does not mean producing a policy document before coding and leaving it on a shelf. It means translating legitimate purpose, authority, and risk tolerance into executable properties of the system:

- schemas that reject invalid evidence references;
- tool scopes that enforce least privilege;
- approval tokens that models cannot mint;
- append-only, tamper-evident traces;
- budgets and timeouts;
- versioned prompts and policies;
- evaluation gates in deployment pipelines;
- kill switches and credential revocation.

Governance principle

The right to generate a recommendation is not the right to execute it. Permission to query one source is not permission to search all sources. Authority to investigate is not authority to accuse or punish. A mature agentic architecture makes these distinctions technically enforceable and institutionally contestable.

10

Implications, Limits, and the Road Ahead

Chapter compass

Argument. Generative AI evolved by turning language from an output medium into a medium for task specification, intermediate computation, tool control, and coordinated action.
Reader outcome. Articulate both the continuity of the underlying mechanism and the new scientific, engineering, and institutional obligations introduced at each stage.

10.1 The evolution summarized

The three experiments now form a single argument about how capability is assembled and how obligation expands with it.

Table 10.1: The complete progression

Stage	Holmes experiment	Added capability	New obligation
Predictive	small Transformer on stories	learns distributional style and continuation	provenance, split integrity, memorization checks
Adaptive	pretrained instruction base	interprets task contracts and examples	preference transparency, prompt robustness
Reasoning	QLoRA on structured cases	exposes evidence, hypotheses, contradictions, tests	process validation, counterfactual and OOD evaluation
Tool-using	bounded simulated log and evidence tools	obtains external observations within the case environment	access control, schema checks, data minimization
Agentic	orchestrated multi-role loop	plans, acts, observes, revises, stops	trajectory audit, approvals, recovery, accountability

The transition is cumulative:

$$\begin{aligned} \text{Generator} &= \text{model} + \text{decoding}, \\ \text{Reasoner} &= \text{generator} + \text{deliberative training/search} + \text{verification}, \\ \text{Agent} &= \text{reasoner} + \text{goal} + \text{tools} + \text{state} + \text{control loop}, \\ \text{Governed agent} &= \text{agent} + \text{authority} + \text{gates} + \text{audit} + \text{accountability}. \end{aligned} \tag{10.1}$$

This is a conceptual decomposition, not a claim that every product follows one implementation path. Its value is diagnostic. When a system succeeds or fails, the decomposition helps identify whether the decisive factor lies in the model, training signal, inference procedure, tool layer, state, control logic, authority, or surrounding institution.

10.2 Five misconceptions the ladder corrects

10.2.1 "Prediction and reasoning are different species"

Reasoning models remain generative models. Their advances come from data, rewards, search, verification, and inference budgets that organize the generative process. The continuity does not diminish the advance; it makes the advance intelligible.

10.2.2 "A long rationale proves deep reasoning"

Length is not validity. Reliable performance requires tests that respond to evidence, counterfactuals, and executable constraints. Explanations should expose decision-relevant commitments, not serve as ornamental narration.

10.2.3 "Tools make a model intelligent"

Tools extend reach and precision. They can also introduce stale data, faulty APIs, excessive permissions, and execution errors. Intelligence lies partly in choosing, parameterizing, interpreting, and declining tools appropriately.

10.2.4 "Multiple agents are necessarily superior"

Multi-agent systems are justified when specialization, parallelism, access separation, or adversarial review produces measurable gains. Otherwise, they may be a costly way for correlated models to agree with themselves.

10.2.5 "Autonomy is the final objective"

The objective is legitimate value under acceptable risk. More autonomy is useful only when the expected benefit of delegation exceeds the added cost of uncertainty, oversight, and consequence. In many domains the best architecture is a highly capable adviser operating under sharply limited authority.

10.3 Capabilities still missing from the demonstration

The notebook-scale Holmes system remains intentionally incomplete.

- **Scale:** six curated cases and synthetic curricula cannot establish broad investigative competence.
- **Domain validity:** fictional mysteries do not substitute for forensic, legal, cybersecurity, or financial expertise.
- **Calibration:** model confidence fields are not guaranteed probabilities.
- **Faithfulness:** structured output does not expose every internal causal factor.
- **Adversarial resilience:** prompt injection and poisoned evidence require dedicated testing.
- **World grounding:** the executed agent used bounded simulated tools; deployment would require live typed connectors, production identities, permission enforcement, realistic failure conditions, and controlled environments.
- **Institutional legitimacy:** technical performance cannot determine lawful authority or due process.

These limitations do not invalidate the pedagogical progression, but they sharply limit empirical generalization. They define the evidentiary boundary between a transparent demonstration and a defensible deployment claim.

10.4 A research agenda

10.4.1 Build a richer reasoning corpus

Create hundreds or thousands of evidence-grounded cases with multiple annotators, explicit ambiguity, adversarial distractors, alternative valid conclusions, and domain transfer. Record inter-annotator disagreement rather than forcing false certainty.

10.4.2 Develop hybrid verification

Combine deterministic schema checks, retrieval-based evidence validation, causal or probabilistic models, code execution, and expert review. Different claims require different verifiers; there is no universal truth detector.

10.4.3 Measure adaptive computation

Study whether the system allocates more search to difficult cases, stops early on easy ones, and uses additional compute productively rather than merely producing longer traces.

10.4.4 Create a mystery-agent simulator

Build an environment in which actions reveal controlled evidence, consume budgets, and sometimes return noise or deception. The agent can then be evaluated on information seeking, recovery, and policy compliance without affecting real people.

10.4.5 Run component and governance ablations

Measure the marginal value of the reasoner, generator, critic, verifier, memory, approval gate, and audit interface. Treat governance controls as testable system components, not external paperwork.

10.4.6 Transfer beyond Holmes

The strongest evidence of method learning would be transfer to novel structures:

- financial anomaly investigation;
- cybersecurity incident triage;
- scientific hypothesis testing;
- operational root-cause analysis;
- legal issue spotting under explicit evidentiary limits.

Each domain requires its own experts, tools, risks, and authorization model. The transferable object is the abstract cycle of observation, hypothesis, prediction, test, update, and calibrated conclusion.

10.5 The deeper conceptual lesson

The deepest conceptual shift is that generative AI has turned language from an output medium into a medium of control.

At the first rung, language is what the model generates.

At the reasoning rung, language also represents intermediate problems, hypotheses, critiques, and checks.

At the agentic rung, language can specify goals, describe tools, encode policies, communicate between specialists, and summarize observations. Natural language becomes part of the software architecture.

This is why modern systems can feel qualitatively different while retaining next-token generation at the core. They use language not only to describe the world, but also to organize computation about it, coordinate components, invoke formal interfaces, and—through tools—intervene in it.

10.6 Conclusion

The progression from next-token generation to reasoning and agency is neither magic nor merely a change in marketing vocabulary. It is a sequence of concrete scientific, computational, and institutional additions.

The first Holmes model learns the statistical texture of detective fiction. It predicts what continuation belongs in the neighborhood of the prompt. The second model is adapted to an evidentiary discipline. It must enumerate observations, preserve alternatives, expose contradictions, identify useful tests, and calibrate a conclusion. The third system gives these models specialized roles inside a governed loop. It can acquire authorized observations, update a case, challenge itself, and produce a provisional judgment.

Every ascent creates both power and obligation. Prediction creates the risk of plausible falsehood. Reasoning creates the risk of persuasive invalidity. Tool use creates the risk of incorrect execution and inappropriate access. Agency creates the risk of compounding action across time. Accordingly, the architecture of intelligence must be accompanied by an architecture of authority.

The most important lesson is therefore not that generative AI has finally "become autonomous." It is that autonomy is constructed. It emerges when probabilistic models are coupled to objectives, intermediate computation, tools, state, feedback, and delegated permissions. Because autonomy is constructed, its boundaries are design choices. They can—and must—be evaluated, enforced, audited, contested, and governed.

The progression ladder

A generator asks: What could come next?
A reasoner asks: What follows under the evidence and constraints?
An agent asks: What should be done next to advance the goal?
A governed institution asks: What may this system legitimately do, under whose authority, with what proof and recourse?

A

Technical Companion

Chapter compass

Purpose. Translate the conceptual ladder into inspectable experimental objects: training loops, schemas, agent control flow, comparison conditions, evidence records, and stopping rules.
Use. Treat the pseudocode as a design specification to be adapted and tested, not as production-ready implementation.

A.1 Experiment map

Table A.1: Notebook-to-monograph mapping

Artifact	Principal mechanism	Monograph role
Next-sentence Transformer notebook	from-scratch word-level causal Transformer trained on five public-domain works	demonstrates Rung One: probabilistic continuation
Holmesian reasoning notebook	QLoRA adaptation of a small instruction model using curated and procedural reasoning cases	demonstrates Rung Three: evidence-grounded protocol learning
Autonomous mystery notebook and reference architecture	executed simplified orchestrator with model roles, ledger, simulated tools, state, gates, trace, and report	demonstrates bounded tool use and agency while specifying the fuller Rungs Four and Five architecture

A.2 Causal language-model pseudocode

```
for batch in train_loader:
   input_ids = batch[:, :-1]
   target_ids = batch[:, 1:]
```

```
   logits = model(input_ids, causal_mask=True)
   loss = cross_entropy(
      logits.reshape(-1, vocab_size),
      target_ids.reshape(-1)
   )

   optimizer.zero_grad()
   loss.backward()
   clip_grad_norm_(model.parameters(), max_norm=1.0)
   optimizer.step()
```

The shift by one position implements next-token learning. At generation time, the selected token is appended to the context and the procedure repeats.

A.3 Reasoning adaptation pseudocode

```
base = load_instruct_model(load_in_4bit=True)
model = attach_lora(
   base,
   rank=R,
   target_modules=["q_proj", "k_proj", "v_proj", "o_proj"]
)

for batch in reasoning_loader:
   logits = model(batch["input_ids"])
   loss = masked_cross_entropy(
      logits,
      batch["labels"],
      assistant_token_mask=batch["target_mask"]
   )
   loss.backward()
   optimizer.step()
```

Only the adapter parameters receive updates. The target response follows the reasoning schema. Case-level splitting occurs before tasks are expanded.

A.4 Governed agent-loop pseudocode

```
state = initialize_case(user_submission, policy)

while not state.done:
   evidence = custodian.normalize(state)
   analysis = reasoner.analyze(evidence, state.open_questions)
   scenarios = generator.propose(evidence, label="SPECULATIVE")
   critique = critic.challenge(analysis, scenarios)
   verified = verifier.check(evidence, analysis, critique)

   if verified.requires_revision:
      state.record_rejection(verified)
      state = orchestrator.replan(state)
```

```
      continue

   proposed_test = orchestrator.select_test(
      verified,
      budget=state.remaining_budget
   )

   decision = authorization_gate.evaluate(
      proposed_test,
      policy=state.policy,
      human_approver=state.owner
   )

   if decision.status == "APPROVED":
      observation = tool_broker.execute(decision.scoped_action)
      state.append_observation(observation)
   elif decision.status == "ESCALATE":
      state.pause_for_human(decision.reason)
   else:
      state.record_denial(decision.reason)

   state.done = orchestrator.should_stop(state)

return report_agent.write(state.accepted_evidence, state.audit_trace)
```

The pseudocode is intentionally explicit about the authorization gate. A model proposal cannot call the tool directly.

A.5 Suggested experimental matrix

Table A.2: A rigorous comparative experiment

Condition	Generator	Reasoning	Agency
A	from-scratch Holmes LM	none	none
B	base instruction model	prompt-only schema	none
C	base instruction model	QLoRA schema	none
D	Holmes LM for scenarios	QLoRA reasoner + verifier	fixed workflow
E	Holmes LM for scenarios	QLoRA reasoner + critic	bounded tool-using agent
F	ablated	single general model	same tools and policies as E

Comparisons C versus B measure adapter value. D versus C measures orchestration without dynamic agency. E versus D measures adaptive tool use. E versus F measures whether specialization justifies multi-agent complexity.

A.6 Reproducibility checklist

- Record package and hardware versions.
- Fix random seeds, while repeating runs across several seeds.
- Hash source texts and derived datasets.
- Preserve exact train, validation, and OOD case identifiers.
- Save tokenizer, base model identifier, adapter, and configuration.
- Record prompts, schema versions, and decoding parameters.
- Export training history and evaluation outputs.
- Store verifier rules and preference-pair corruption functions.
- Version tool descriptions, policies, and authorization matrices.
- Report failed runs and deviations from the protocol.

A.7 Stopping conditions

An agent should stop when any of the following holds:

$$\text{Stop} = G_{\text{satisfied}} \vee B_{\text{exhausted}} \vee R_{\text{exceeded}} \vee A_{\text{required}} \vee U_{\text{irreducible}} \vee F_{\text{system}}, \tag{A.1}$$

where the terms denote goal satisfaction, budget exhaustion, excessive risk, required authorization, irreducible uncertainty, and system failure. “Continue until certain” is not a valid stopping policy; many investigations cannot eliminate uncertainty.

A.8 A minimal evidence object

```
{
  "evidence_id": "E3",
  "claim": "The document was opened remotely at 20:32.",
  "status": "LOGGED_EVENT",
  "source": {
    "system": "document_management",
    "record_id": "opaque-id",
    "collected_at": "2026-06-18T22:10:00-06:00"
  },
  "integrity": {
    "sha256": "record-hash",
    "collector": "authorized-collector"
  },
  "limitations": [
```

```
    "A file-open event does not identify the human operator."
  ]
}
```

The `limitations` field prevents a common inferential collapse: equating a credential or device identifier with a verified person.

A.9 Executable Bayesian and risk exercises

The following functions reproduce the worked calculations in Chapters 4 and 8. They deliberately expose every assumption; replacing a prior, likelihood, cost, or weight changes the result immediately. The calculations are auditable but not calibrated: their pedagogical purpose is to make the equations falsifiable by allowing a reader to reproduce the ranking, alter the assumptions, and identify which premise changes the decision.

```
from math import log2
def entropy(p):
    if p in (0.0, 1.0):
        return 0.0
    return -p * log2(p) - (1 - p) * log2(1 - p)
def binary_update(prior, p_hit_h1, p_hit_h0):
    p_hit = p_hit_h1 * prior + p_hit_h0 * (1 - prior)
    post_hit = p_hit_h1 * prior / p_hit
    post_miss = (1 - p_hit_h1) * prior / (1 - p_hit)
    return p_hit, post_hit, post_miss
def information_gain(prior, p_hit_h1, p_hit_h0):
    p_hit, post_hit, post_miss = binary_update(
        prior, p_hit_h1, p_hit_h0
    )
    expected = (
        p_hit * entropy(post_hit)
        + (1 - p_hit) * entropy(post_miss)
    )
    return entropy(prior) - expected
device_eig = information_gain(0.60, 0.80, 0.20)
inventory_eig = information_gain(0.60, 0.60, 0.40)
print(round(device_eig, 3)) # 0.268 bits
print(round(inventory_eig, 3)) # 0.028 bits
def risk_objective(reward, compute, human, harm, violation,
                   lam=0.20, mu=0.50, nu=2.00, xi=3.00):
    return (
        reward - lam * compute - mu * human
        - nu * harm - xi * violation
    )
scoped = risk_objective(0.70, 0.10, 0.05, 0.05, 0)
prohibited = risk_objective(0.95, 0.15, 0.05, 0.12, 1)
print(round(scoped, 3)) # 0.555
print(round(prohibited, 3)) # -2.345
```

B

Glossary

Chapter compass

Purpose. Provide operational definitions for terms that are often used loosely across research, engineering, policy, and product discourse.
Reading principle. Interpret each term according to the observable system property it denotes, not according to anthropomorphic implication.

Abduction
Inference to the best available explanation. It generates hypotheses that could account for observed evidence.

Adaptation
Changing model behavior for a task or domain through weight updates, adapters, prompting, demonstrations, retrieval, or other conditioning mechanisms. These mechanisms differ in persistence and in whether model parameters change.

Action trajectory
The time-ordered sequence of states, proposed actions, executed actions, observations, and updates produced by an agent.

Agent
A goal-directed software system in which a model controls part of a stateful workflow and can select or invoke tools under constraints.

Agentic architecture
The surrounding system that gives one or more models roles, goals, tools, state, interaction rules, guardrails, and stopping conditions.

Attention
A learned mechanism that lets each token representation combine information from other relevant token positions.

Audit bundle
A versioned collection of manifests, prompts, evidence records, tool calls, deci-

sions, approvals, errors, and outputs needed to reconstruct and assess a system run.

Authorization gate
A deterministic or human-controlled checkpoint that decides whether a proposed action is permitted.

Calibration
The correspondence between expressed confidence and empirical correctness over repeated cases.

Causal faithfulness
The degree to which a stated reasoning trace reflects intermediate content that materially influenced the model's answer. Faithfulness is distinct from logical validity and factual correctness.

Causal language model
A model trained to predict each token using only preceding tokens.

Chain of thought
An intermediate sequence used to decompose or work through a problem. Visible chain-of-thought text should not automatically be treated as a faithful explanation of internal computation.

Context window
The finite sequence of tokens available to a model for a particular inference.

Counterfactual test
An evaluation that changes or removes evidence to determine whether the model's conclusion responds appropriately.

Cross-entropy
The standard loss for next-token prediction, penalizing low probability assigned to the observed target token.

Deduction
Inference from premises and rules to a conclusion that must hold if the premises and rules are valid.

Deliberation
Allocation of intermediate computation to decomposition, search, hypothesis comparison, checking, critique, or revision before a final answer or action is selected.

Evidence grounding
The requirement that material claims be traceable to admitted observations or sources.

Foundation model
A broadly trained model adaptable to many downstream tasks (Bommasani et al., 2021).

Generative AI
Models or systems that generate derived content such as text, images, audio, video, or code from learned data distributions.

Guardrail
A control that detects, blocks, modifies, constrains, or escalates model inputs, outputs, or actions.

Hallucination
Generated content that is unsupported, false, or inconsistent with the available source or world, despite often appearing fluent.

In-context learning
Task adaptation from instructions or demonstrations supplied in the current prompt, without changing model weights.

Instruction tuning
Supervised adaptation on pairs of user instructions and desired responses.

Least privilege
The principle that each component receives only the data access and action authority necessary for its assigned task, for no longer than required.

LoRA
Low-Rank Adaptation: a parameter-efficient method that learns low-rank updates while freezing base-model weights.

Memory
Persistent or semi-persistent state used by an agent beyond the immediate model context.

Model
A parameterized function that maps input context to an output distribution. A model alone is not an agent.

Next-token prediction
Estimating $p(x_t \mid x_{<t})$, the probability of the next token conditional on preceding tokens.

Outcome supervision
Feedback based on the quality or correctness of a final result.

Perplexity
The exponential of average cross-entropy loss, used to summarize predictive uncertainty under a fixed evaluation setup.

Post-training
Adaptation performed after broad pretraining to shape instruction following, preferences, reasoning behavior, tool use, safety, or domain-specific performance.

Process supervision
Feedback applied to intermediate steps rather than only the final answer.

Provenance
Information about the origin, collection, transformation, custody, and integrity of data or evidence.

QLoRA
Quantized LoRA: adapter training through a frozen, low-bit representation of the base model.

Reasoning model
Operationally, a model trained or configured to allocate intermediate computation to multi-step problem solving, search, critique, or verification.

Retrieval-augmented generation
Supplying externally retrieved passages to a model at inference time. Retrieval normally changes context, not model weights.

Risk gate
A policy or human checkpoint that approves, narrows, denies, or escalates a proposed action according to its consequence and authority requirements.

Sociotechnical system
A system whose outcomes arise jointly from models, software, data, tools, people, policies, organizations, incentives, and affected communities.

Test-time compute
Computation allocated during inference to sampling, search, verification, tool use, simulation, or revision rather than to model training.

Structured output
A model response constrained to a defined schema so that required fields, types, references, and bounds can be checked programmatically.

Tool
A typed interface that lets an agent obtain information or perform an operation outside the model.

Trajectory
The ordered sequence of states, decisions, actions, observations, and updates through which a reasoning process or agent reaches an outcome.

Verifier
A model, program, rule set, executor, or human that evaluates a claim, step, output, or trajectory against independent criteria.

Workflow
A sequence or graph of steps whose control flow is mainly predetermined by software.

Bibliography

Chloe Autio, Reva Schwartz, Jesse Dunietz, Shomik Jain, Martin Stanley, Elham Tabassi, Patrick Hall, and Kamie Roberts. Artificial intelligence risk management framework: Generative artificial intelligence profile. Technical Report NIST AI 600-1, National Institute of Standards and Technology, 2024. URL https://doi.org/10.6028/NIST.AI.600-1.

Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, Erik Brynjolfsson, Shyamal Buch, Dallas Card, Rodrigo Castellon, Niladri Chatterji, Annie Chen, Kathleen Creel, Jared Quincy Davis, Dorottya Demszky, Chris Donahue, Moussa Doumbouya, Esin Durmus, Stefano Ermon, John Etchemendy, Kawin Ethayarajh, Li Fei-Fei, Chelsea Finn, Trevor Gale, Lauren Gillespie, Karan Goel, Noah Goodman, Shelby Grossman, Neel Guha, Tatsunori Hashimoto, Peter Henderson, John Hewitt, Daniel E. Ho, Jenny Hong, Kyle Hsu, Jing Huang, Thomas Icard, Saahil Jain, Dan Jurafsky, Pratyusha Kalluri, Siddharth Karamcheti, Geoff Keeling, Fereshte Khani, Omar Khattab, Pang Wei Koh, Mark S. Krass, Ranjay Krishna, Rohith Kuditipudi, Ananya Kumar, Faisal Ladhak, Mina Lee, Tony Lee, Jure Leskovec, Isabelle Levent, Xiang Lisa Li, Xuechen Li, Tengyu Ma, Ali Malik, Christopher D. Manning, Suvir P. Mirchandani, Eric Mitchell, Zanele Munyikwa, Suraj Nair, Avanika Narayan, Deepak Narayanan, Benjamin Newman, Allen Nie, Juan Carlos Niebles, Hamed Nilforoshan, Julian Nyarko, Giray Ogut, Laurel Orr, Isabel Papadimitriou, Joon Sung Park, Chris Piech, Eva Portelance, Christopher Potts, Aditi Raghunathan, Rob Reich, Hongyu Ren, Frieda Rong, Yusuf Roohani, Camilo Ruiz, Jack Ryan, Christopher Ré, Dorsa Sadigh, Shiori Sagawa, Keshav Santhanam, Andy Shih, Krishnan Srinivasan, Alex Tamkin, Rohan Taori, Armin W. Thomas, Florian Tramèr, Rose E. Wang, William Wang, Bohan Wu, Jiajun Wu, Yuhuai Wu, Sang Michael Xie, Michihiro Yasunaga, Jiaxuan You, Matei Zaharia, Michael Zhang, Tianyi Zhang, Xikun Zhang, Yuhui Zhang, Lucia Zheng, Kaitlyn Zhou, and Percy Liang. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021. URL https://arxiv.org/abs/2108.07258.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, 2020. URL https://arxiv.org/abs/2005.14165.

Yanda Chen, Joe Benton, Ansh Radhakrishnan, Jonathan Uesato, Carson Denison, John Schulman, Arushi Somani, Peter Hase, Misha Wagner, Fabien Roger, Vlad Mikulik, Samuel R. Bowman, Jan Leike, Jared Kaplan, and Ethan Perez. Reasoning models don't always say what they think. *arXiv preprint arXiv:2505.05410*, 2025. URL https://arxiv.org/abs/2505.05410.

DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025. URL https://arxiv.org/abs/2501.12948.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://arxiv.org/abs/2305.14314.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL https://arxiv.org/abs/2203.15556.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL https://arxiv.org/abs/2106.09685.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020. URL https://arxiv.org/abs/2001.08361.

Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, Evan Hubinger, Jackson Kernion, Kamilė Lukošiūtė, Karina Nguyen, Newton Cheng, Nicholas Joseph, Nicholas Schiefer, Oliver Rausch, Robin Larson, Sam McCandlish, Sandipan Kundu,

Saurav Kadavath, Shannon Yang, Thomas Henighan, Timothy Maxwell, Timothy Telleen-Lawton, Tristan Hume, Zac Hatfield-Dodds, Jared Kaplan, Jan Brauner, Samuel R. Bowman, and Ethan Perez. Measuring faithfulness in chain-of-thought reasoning. *arXiv preprint arXiv:2307.13702*, 2023. URL https://arxiv.org/abs/2307.13702.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, 2020. URL https://arxiv.org/abs/2005.11401.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2023. URL https://arxiv.org/abs/2305.20050.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. URL https://arxiv.org/abs/2307.03172.

Qing Lyu, Shreya Havaldar, Adam Stein, Li Zhang, Delip Rao, Eric Wong, Marianna Apidianaki, and Chris Callison-Burch. Faithful chain-of-thought reasoning. In *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*, pages 305–329, 2023. doi: 10.18653/v1/2023.ijcnlp-main.20. URL https://aclanthology.org/2023.ijcnlp-main.20/.

OpenAI. Learning to reason with LLMs. OpenAI, 2024. URL https://openai.com/index/learning-to-reason-with-llms/. Published September 12, 2024.

OpenAI. A practical guide to building agents. OpenAI, 2025. URL https://cdn.openai.com/business-guides-and-resources/a-practical-guide-to-building-agents.pdf.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL https://arxiv.org/abs/2203.02155.

Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4 edition, 2021.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://arxiv.org/abs/2302.04761.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik R. Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://arxiv.org/abs/2303.11366.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024. URL https://arxiv.org/abs/2408.03314.

Elham Tabassi. Artificial intelligence risk management framework (AI RMF 1.0). Technical Report NIST AI 100-1, National Institute of Standards and Technology, 2023. URL https://doi.org/10.6028/NIST.AI.100-1.

Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://arxiv.org/abs/2305.04388.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://arxiv.org/abs/1706.03762.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2203.11171.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL https://arxiv.org/abs/2201.11903.

Michael Wooldridge. *An Introduction to MultiAgent Systems*. John Wiley & Sons, 2 edition, 2009.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023. URL https://arxiv.org/abs/2308.08155.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023a. URL https://arxiv.org/abs/2305.10601.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023b. URL https://arxiv.org/abs/2210.03629.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL https://arxiv.org/abs/2203.14465.

Colophon

This monograph was typeset in Latin Modern Roman, Source Sans Pro, and Source Code Pro. The source is compatible with pdfLaTeX and LuaLaTeX. Its restrained blue palette is inspired by Cambridge editorial design without representing an official University publication.

The progression ladder, system architectures, tables, equations, and fictional case were created specifically for this edition. References emphasize primary research papers, official technical publications, and established textbooks. The diagrams are rendered as vector graphics for print clarity.

Prediction creates possibilities.
Reasoning disciplines them.
Agency gives them consequences.
Governance gives those consequences legitimate boundaries.